

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
«ВЫСШАЯ ШКОЛА ЭКОНОМИКИ»

Московский институт электроники и математики им. А.Н. Тихонова

Шишулин Дмитрий Иванович

**ОЦЕНКА ВОЗМОЖНОСТИ ПОСТРОЕНИЯ ARX ПОДСТАНОВКИ СО
СХОЖИМИ КРИПТОГРАФИЧЕСКИМИ СВОЙСТВАМИ С
ПОДСТАНОВКАМИ ШИФРА AES**

Выпускная квалификационная работа
по специальности 10.05.01 «Компьютерная безопасность»
студента образовательной программы специалитета
«Компьютерная безопасность»

Студент

Научный руководитель
к.ф.-м.н., доцент кафедры Компьютерной
Безопасности МИЭМ НИУ ВШЭ

_____ Д.И. Шишулин
подпись И.О. Фамилия

_____ А.А. Дмух
подпись И.О. Фамилия

Рецензент

Старший научный сотрудник
лаборатории математических проблем
теоретической криптографии Академии
криптографии Российской Федерации.

_____ Д.О. Пасько
И.О. Фамилия

Москва 2026

Аннотация

8-битные подстановки являются ключевым элементом современных блочных шифров и хэш-функций, определяя их устойчивость к линейному, дифференциальному и алгебраическому криптоанализу. Традиционно в промышленных стандартах используются табличные S-Box-подстановки, однако их применение в низкоресурсных системах связано с ограничениями по памяти и вычислительной мощности. В этой связи возрастающий интерес вызывают ARX-конструкции, основанные исключительно на операциях сложения по модулю 2^w (Add), циклического сдвига на заданное количество позиций (Rotate) и покоординатного сложения по модулю 2 (XOR), которые позволяют отказаться от табличного задания, что потенциально дает преимущества при их реализации в низкоресурсной криптографии.

Цель данной работы - исследовать криптографические свойства 8-битных ARX-подстановок и сопоставить полученные результаты с эталонными табличными подстановками, такими как S-Box AES и ГОСТ Р 34.12-2015 «Кузнечик». В отличие от классических подходов, ARX-подстановки обладают структурной простотой и аппаратной эффективностью, однако их криптографические характеристики требуют тщательного анализа, поскольку пространство параметров велико, а хорошие параметризации встречаются крайне редко.

В рамках исследования использована методика полного перебора и ускоренного вычисления разностной, линейной и алгебраической характеристик ARX-подстановок с использованием оптимизаций на CPU (OpenMP, SIMD) и GPU (CUDA). Проведён масштабный экспериментальный анализ криптографических свойств, выявлены параметризации обеспечивающие наилучшие криптографические характеристики. Полученные результаты позволят определить пригодность ARX-подстановок для использования в шифровании и хэш-функциях.

Annotation

Iterated ARX (Add–Rotate–XOR) constructions offer a lightweight alternative to table-based S-boxes, particularly in constrained environments where memory usage and timing determinism are critical. This work investigates whether multi-round ARX templates can generate 8-bit bijective substitutions with differential, linear, and algebraic properties comparable to established designs such as the AES S-box. The study performs exhaustive enumeration of parameter sets, verifies bijectivity, and computes exact cryptographic metrics using optimized CPU and GPU implementations. The results include full distributions of differential probabilities, linear biases, and algebraic degrees, as well as identification of Pareto-optimal candidates. While a small number of configurations approach AES-level characteristics, such instances are shown to be extremely rare. The findings provide a reproducible basis for evaluating ARX-based S-boxes and guiding targeted searches for secure tableless nonlinear components.

Содержание

Введение.....	3
Глава 1.....	8
Основные определения и обозначения	8
Разностная характеристика	10
Линейная характеристика.....	13
Алгебраическая степень	15
Выбранная ARX-конструкция	18
Описание одной итерации ARX-преобразования.....	18
Композиция итераций	19
Параметризация.....	20
Методы вычисления.....	21
Грубые методы вычисления характеристик	22
Эффективные методы вычисления характеристик	27
Реализация вычислений характеристик.....	36
Реализация эталонного кода на Си	37
Оптимизация вычислений с использованием OpenMP и SIMD	40
Сравнение производительности эталонного кода и оптимизированной программы.....	44
Реализация вычислений на CUDA	48
Сравнение производительности различных вариантов программ	53
Валидация полученных характеристик.....	55
Глава 2. Экспериментальная часть и результаты	57
Описание экспериментов.....	58

Сравнение реализаций	59
Результаты вычислений	60
Сравнение с AES и ГОСТ «Кузнечик»	66
Выводы	72
Библиографический список	74
Приложение	
Листинг 1. Эталонная программа на языке C. (etalon.c)	1
Листинг 2. Программа, оптимизированная с помощью SIMD и OpenMP (etalon_opt.c)	5
Листинг 3. Программа, реализованная на CUDA. (cuda_version.cu)	11
Листинг 4. Скрипт на языке Python для валидации характеристик. (sage.py)	21

Введение

Подстановки на множестве 8-битных векторов являются одним из базовых инструментов симметричной криптографии. Их использование связано с необходимостью введения нелинейности (реализация Шенноновского принципа перемешивания) в алгоритмы шифрования и хэширования. Линейные операции, такие как сложение по модулю 2, циклические сдвиги и матричные линейные преобразования, сами по себе не обеспечивают достаточной стойкости. Нелинейные компоненты препятствуют построению простых алгебраических моделей и снижают эффективность методов линейного и дифференциального криптоанализа.

Важность 8-битных подстановок подтверждается примерами государственных стандартов. В алгоритме AES используется фиксированная биекция на множестве 8-битных слов, известная как S-box-подстановка. S-box-подстановка представляет собой байтовую подстановку, выполняющую функцию основного нелинейного блока в архитектуре SPN-шифров [1]. В практических реализациях S-box-подстановка может быть представлена либо табличным заданием (lookup-table), либо как вычисляемая функция. Назначение S-box-подстановки – нарушать линейные и алгебраические зависимости, возникающие при применении рассеивающих линейных слоёв, и тем самым повышать устойчивость к дифференциальному и линейному криптоанализу. Подстановка AES характеризуется высокой алгебраической степенью λ_π , достаточно низким значением разностной характеристикой r_π и достаточно малой величиной линейной характеристики δ_π . Конструкция S-box-подстановки AES основана на инверсии в поле $GF(2^8)$ с последующим аффинным преобразованием, что обеспечивает прозрачную математическую модель для анализа и даёт вышеописанные криптографические свойства [2]. Совокупность этих свойств обеспечивает устойчивость AES к основным видам атак, что позволило алгоритму стать международным стандартом криптографической защиты информации.

В отечественном стандарте «Кузнечик» (ГОСТ Р 34.12-2015) также применяется 8-битная подстановка, прошедшая строгую проверку на соответствие криптографическим требованиям [3]. Она выполняет ту же функцию, что и S-box-подстановка в AES, являясь основным источником нелинейности и дополняя линейные рассеивающие операции. Таким образом, в разных криптографических стандартах именно 8-битные подстановки рассматриваются как ключевой элемент обеспечения стойкости.

Значение подобных преобразований проявляется и в хэш-функциях. В семействах SHA-2 и SHA-3 нелинейность достигается комбинацией побитовых операций, сложения по модулю и циклических сдвигов[4]. Несмотря на различие в реализации, принцип использования компактных нелинейных преобразований остаётся фундаментальным. Они обеспечивают диффузию, то есть быстрое распространение изменений входных данных по выходу, и тем самым препятствуют построению предсказуемых зависимостей.

Актуальность исследования 8-битных подстановок определяется несколькими факторами. Во-первых, они являются универсальным инструментом, применимым как в блочных шифрах, так и в хэш-функциях. Во-вторых, их свойства можно формально описывать и проверять, что делает возможным строгий математический анализ. В-третьих, именно эти преобразования определяют уровень стойкости алгоритмов к основным видам криптоанализа. Наконец, развитие альтернативных подходов, таких как ARX-подстановки, открывает перспективы создания новых конструкций, которые могут заменить табличные S-box-подстановки в условиях ограниченных ресурсов или в bit-slice реализациях.

Табличные S-box-подстановки остаются распространённым решением для введения нелинейности в блочные шифры и хэш-функции. В то же время их применение в низкоресурсных системах сопряжено с рядом практических ограничений. Во-первых, табличная реализация требует хранения 256 байт для каждой 8-битной подстановки и дополнительных вспомогательных структур

при оптимизациях. Для микроконтроллеров с ограниченным объёмом флеш-памяти и ОЗУ это приводит к заметному росту размера прошивки и к необходимости компромиссов между кодом и данными. Во-вторых, обращения к таблицам создают зависимость времени выполнения от состояния кэша и памяти. На общих процессорах и встраиваемых платформах это даёт вектор для побочных каналов, в частности для кэш-тайминг-атак и атак по энергопотреблению, что усложняет достижение реализации в режиме constant-time [5][6]. В-третьих, аппаратная реализация таблиц требует либо выделенных блоков памяти, либо значительной логики для вычисления значений на лету, что может увеличивать площадь кристалла, энергопотребление и сложность верификации в условиях жёстких аппаратных ограничений.

В ответ на эти ограничения в литературе и практике получили распространение ARX-подстановки. ARX-конструкции используют только операции сложения по модулю 2^w , циклические сдвиги и XOR. Их ключевое преимущество для низкоресурсных платформ состоит в отсутствии таблиц, то есть все операции выполняются арифметически и аппаратно поддерживаются большинством процессоров и микроконтроллеров. Это уменьшает требования к памяти, упрощает код и облегчает достижение детерминированного времени выполнения, что снижает риск реализации уязвимостей, связанных с кэшем и ветвлениями. ARX-операции хорошо масштабируются на SIMD- и bit-slice-подходы, что позволяет эффективно распараллеливать обработку и повышать пропускную способность без увеличения объёма таблиц.

Проблематика выбора между табличными S-box-подстановками и ARX-подстановками сводится к поиску баланса между практическими ограничениями платформы и строгими криптографическими требованиями. В условиях ограниченных ресурсов ARX-подходы представляют собой привлекательную альтернативу, но их применение должно сопровождаться

систематическим поиском и верификацией подстановок по критериям стойкости.

Для объективной оценки кандидатов на роль 8-битной подстановки используются формальные метрики, они же представляют из себя критерии стойкости. Формальными метриками для подстановок являются их криптографические свойства, среди которых алгебраическая степень, разностная характеристика и линейная характеристика. Далее даются краткие пояснения их значения в контексте выбора и валидации подстановок.

Алгебраическая степень λ_π - степень многочлена Жегалкина. Параметр отражает минимальную по ненулевым линейным комбинациям сложность булевых функций (степень нелинейности нетривиальной линейной комбинации координатных функций подстановки), порождаемых подстановкой и её обратной. Высокая алгебраическая степень затрудняет построение алгебраических уравнений низкой степени и снижает эффективность алгебраического и дифференциального анализа. Разностная характеристика p_π характеризует наименьшую вероятность дифференциального перехода. Низкое значение p_π уменьшает вероятность успешного применения дифференциального криптоанализа и повышает требуемое число текстов для атаки [7]. Линейная характеристика δ_π - определяет насколько успешно можно линейно аппроксимировать, малая величина δ_π снижает эффективность линейного криптоанализа и увеличивает сложность восстановления ключа по линейным приближениям [8].

Эти три характеристики образуют многокритериальную систему оценки. В реальных задачах улучшение одной метрики может приводить к ухудшению другой, поэтому выбор подстановки требует компромисса, заданного требованиями реализации и моделью угроз. Для низкоресурсных платформ приоритеты часто таковы, что требуется минимизировать разностную характеристику p_π и линейную характеристику δ_π при приемлемой

алгебраической степени λ_π , поскольку низкие значения первых двух параметров напрямую уменьшают практическую эффективность дифференциальных и линейных атак. В то же время для долгосрочной стойкости важно, чтобы алгебраическая степень не была чрезмерно мала, иначе возрастает риск алгебраических уязвимостей.

Целью исследования является оценка 8-битных подстановок на основе выбранной ARX-конструкции, пригодных для низкоресурсных платформ, а также сопоставление криптографических свойств полученных подстановок с эталонными табличными S-box-подстановками с точки зрения криптографической стойкости.

Для достижения поставленной цели решаются следующие задачи:

- Изучить научную литературу и актуальные публикации по ARX-подстановкам и зафиксировать формальные определения используемых метрик и методики их вычисления.
- Реализовать вычисление подстановок по выбранной ARX схеме и определить пространство параметров, обеспечивающее разнообразие кандидатов.
- Реализовать вычисление криптографических свойств и отбор наилучших кандидатов.
- Сравнить полученные результаты с эталонными подстановками (AES S-box-подстановка, ГОСТ «Кузнечик»), проанализировать полученные результаты.

Ожидается получить набор ARX-подстановок с документированными криптографическими характеристиками. Результаты могут помочь снизить зависимость от табличных S-box-подстановок в условиях ограниченных ресурсов и повысить устойчивость реализаций к побочным каналам, сохранив при этом требуемый уровень криптографической стойкости.

Глава 1

Основные определения и обозначения

В данном разделе фиксируются базовые понятия и обозначения, которые будут использоваться в дальнейшем изложении. Это необходимо для строгой формализации криптографических характеристик подстановок и для корректного описания алгоритмов их вычисления.

Пусть

$$V_n = \mathbb{F}_2^n$$

- пространство всех двоичных векторов длины n . Элемент $x \in V_n$ записывается как $x = (x_1, x_2, \dots, x_n)$, где $x_i \in \mathbb{F}_2$.

Для $x, y \in V_n$ определим побитовое сложение (сложение по модулю 2)

$$x \oplus y = (x_1 \oplus y_1, \dots, x_n \oplus y_n),$$

где $\oplus$ в правой части означает сложение в поле $\mathbb{F}_2$.

Для $\alpha, x \in V_n$ определим скалярное произведение как

$$\alpha \cdot x = \bigoplus_{i=0}^{n-1} \alpha_i x_i,$$

где произведение $\alpha_i x_i$ берётся в $\mathbb{F}_2$, а суммирование - по модулю 2.

Булевой функцией называется отображение

$$f: V_n \rightarrow \mathbb{F}_2.$$

Полином Жегалкина. Любая булева функция $f(x_1, \dots, x_n)$ может быть единственным образом представлена в виде многочлена над $\mathbb{F}_2$:

$$f(x_1, \dots, x_n) = a \oplus \left(\bigoplus_{1 \leq i_1 \leq \dots \leq i_k \leq n} a_{i_1, \dots, i_k} \cdot x_{i_1} \cdot \dots \cdot x_{i_k} \right),$$

$$a, a_{i_1, \dots, i_k} \in \{0, 1\}, \quad k \in [1, n]$$

где a – свободный член. Алгебраическая степень функции $\deg(f)$ определяется как максимальное число переменных в произведении монома с ненулевым коэффициентом.

Подстановкой называется биективное отображение

$$\pi: V_n \rightarrow V_n.$$

Для любого $x \in V_n$ имеем представление

$$\pi(x) = (f_1(x), f_2(x), \dots, f_n(x)),$$

где $f_i: V_n \rightarrow \mathbb{F}_2$ – булевы функции, называемые координатными. Обратная подстановка обозначается π^{-1} .

Для слова $x \in \{0, 1\}^w$ циклический сдвиг на r позиций влево определяется как

$$\text{rot}_r(x) = (x_{r+1}, x_{r+2}, \dots, x_w, x_1, \dots, x_r).$$

Аналогично определяется сдвиг вправо. Циклический сдвиг является линейной операцией над словами фиксированной длины.

Быстрое преобразование Уолша–Адамара (FWHT, БПФ). Для булевой функции $f: V_n \rightarrow \mathbb{F}_2$ спектр Уолша определяется как

$$W_f(\alpha) = \sum_{x \in V_n} (-1)^{f(x) \oplus (\alpha \cdot x)}.$$

БПФ – алгоритм, позволяющий вычислять все значения $W_f(\alpha)$ за время $O(n2^n)$, что существенно ускоряет построение таблицы линейных приближений [9].

Далее будут даны формальные определения криптографических характеристик, а также их теоретические и практические ограничения.

Разностная характеристика

Для подстановки $\pi: V_n \rightarrow V_n$ и фиксированных $\alpha \in V_n \setminus \{0\}$, $\beta \in V_n$ определим вероятность дифференциального (разностного) перехода как

$$p_{\alpha, \beta}^{\pi} = \Pr_{x \in V_n} [\pi(x \oplus \alpha) \oplus \pi(x) = \beta],$$

где вероятность берётся при равномерном выборе $x \in V_n$. Тогда разностная характеристика подстановки определяется как

$$p_{\pi} = \max_{\alpha, \beta \in V_n \setminus \{0\}} p_{\alpha, \beta}^{\pi}.$$

Малое значение p_{π} означает низкую предсказуемость дифференциальных переходов и высокую устойчивость к дифференциальному (разностному) криптоанализу.

Нижняя граница:

$$p_{\pi} \geq \frac{1}{2^{n-1}}$$

Доказательство: положим

$$N_{\alpha, \beta} = |\{x \in V: \pi(x \oplus \alpha) \oplus \pi(x) = \beta\}|$$

- количество решений уравнения $\pi(x \oplus \alpha) \oplus \pi(x) = \beta$ при фиксированных α и β , это означает число входов x , для которых при фиксированном входном различии α наблюдается выходное различие β .

Рассмотрим теперь уравнение

$$\pi(x \oplus \alpha) \oplus \pi(x) = \beta.$$

Если оно выполняется для некоторого $x = x_1$, то подставляя $x_1 \oplus \alpha$ вместо x , получаем

$$\pi((x_1 \oplus \alpha) \oplus \alpha) \oplus \pi(x_1 \oplus \alpha) = \pi(x_1) \oplus \pi(x_1 \oplus \alpha) = \beta,$$

то есть уравнение выполняется и для $x = x_1 \oplus \alpha$. Таким образом, каждая пара $\{x, x \oplus \alpha\}$ либо не даёт ни одного решения, либо даёт ровно два решения уравнения.

Отсюда следует, что число решений $N_{\alpha,\beta}$ всегда чётно:

$$N_{\alpha,\beta} \in \{0, 2, 4, \dots\}.$$

В частности, если $N_{\alpha,\beta} \neq 0$, то $N_{\alpha,\beta} \geq 2$.

Тогда дифференциальная вероятность имеет вид

$$p_{\alpha,\beta}^{\pi} = \frac{N_{\alpha,\beta}}{2^n} \in \left\{0, \frac{2}{2^n}, \frac{4}{2^n}, \dots\right\}.$$

Следовательно, для любой пары (α, β) с ненулевой дифференциальной вероятностью выполняется

$$p_{\alpha,\beta}^{\pi} \geq \frac{2}{2^n} = \frac{1}{2^{n-1}}.$$

По определению

$$p_{\pi} = \max_{\alpha, \beta \in V_n \setminus \{0\}} p_{\alpha,\beta}^{\pi}.$$

Если предположить, что $p_{\pi} < \frac{1}{2^{n-1}}$, то для всех α и всех β мы должны иметь либо $p_{\alpha,\beta}^{\pi} = 0$, либо $0 < p_{\alpha,\beta}^{\pi} < \frac{1}{2^{n-1}}$. Но мы только что показали, что всякая ненулевая дифференциальная вероятность удовлетворяет неравенству

$$p_{\alpha,\beta}^{\pi} \geq \frac{1}{2^{n-1}},$$

что приводит к противоречию. Следовательно, существует по крайней мере одна пара (α, β) , для которой

$$p_{\alpha,\beta}^{\pi} \geq \frac{1}{2^{n-1}},$$

и, значит,

$$p_{\pi} \geq \frac{1}{2^{n-1}}.$$

Получаем нижнюю границу:

$$\frac{1}{2^{n-1}} \leq p_{\pi}$$

Верхняя граница:

$$p_{\pi} \leq 1.$$

Доказательство: для любой вероятности $p_{\alpha,\beta}^{\pi} \in [0,1]$, следовательно её максимум не превосходит 1. Точность верхней границы достигается для тривиальных (линейных) отображений в более общем классе функций. Для биекций на V_n формально возможно $p_{\pi} = 1$ только при вырожденных структурах, однако верхняя оценка как общая граница остаётся корректной.

Таким образом, разностная характеристика имеет следующие ограничения:

$$\frac{1}{2^{n-1}} \leq p_{\pi} \leq 1$$

Линейная характеристика

Для $\alpha, \beta \in V_n \setminus \{0\}$ определим линейную корреляцию как

$$\delta_{\alpha, \beta}^{\pi} = 2 \Pr_{x \in V_n} [\alpha \cdot x = \beta \cdot \pi(x)] - 1,$$

где $\alpha \cdot x$ и $\beta \cdot \pi(x)$ - скалярные произведения в $\mathbb{F}_2$. Максимальная линейная корреляция подстановки или же линейная характеристика определяется как

$$\delta_{\pi} = \max_{\alpha \in V_n \setminus \{0\}, \beta \in V_n \setminus \{0\}} |\delta_{\alpha, \beta}^{\pi}|.$$

Малое значение δ_{π} свидетельствует о слабой линейной аппроксимируемости подстановки и высокой устойчивости к линейному криптоанализу.

Верхняя и нижняя границы для линейной характеристики:

$$2^{-\frac{n-1}{2}} \leq \delta_{\pi} \leq 1.$$

Доказательство: положим

$$p = \Pr_{x \in V_n} [\alpha \cdot x = \beta \cdot \pi(x)].$$

Тогда по определению $\delta_{\pi}^{\alpha, \beta} = 2p - 1$. Поскольку $p \in [0, 1]$, имеем

$$2p - 1 \in [-1, 1].$$

Таким образом ограничения линейной характеристики сверху имеют вид

$$\delta_{\pi} \leq 1$$

Далее докажем нижнюю границу, доказательство будет строится на основе публикации [10]. Тогда

Пусть $F: \mathbb{F}_{2^n} \rightarrow \mathbb{F}_{2^n}$. Для $a \neq 0, b \neq 0$ определено

$$L_F(a, b) = \{x \in \mathbb{F}_{2^n}: a \cdot x \oplus b \cdot F(x) = 0\}, A_F(a, b) = \#L_F(a, b) - 2^{n-1},$$

и

$$A_F = \sup_{a \neq 0, b \neq 0} |A_F(a, b)|.$$

В лемме 2 статьи показано, что соответствующий Walsh-коэффициент равен $\widehat{O_F}(a, b) = 2A_F(a, b)$. Нормированная линейная корреляция для пары (a, b) равна

$$\delta_{a,b} = \frac{2A_F(a, b)}{2^n} = \frac{A_F(a, b)}{2^{n-1}},$$

откуда

$$\delta_\pi = \frac{A_F}{2^{n-1}}$$

По теореме 4 работы [10] для общего $F: \mathbb{F}_{2^p} \rightarrow \mathbb{F}_{2^q}$ имеет место

$$A_F \geq \frac{1}{2} \left(\frac{3 \cdot 2^q - 2 - 2^{2q-p}}{2^q - 1} \right)^{\frac{1}{2}} \sqrt{(2^p - 1)(2^{p-1} - 1)}.$$

При $p = q = n$ дробь под корнем равна 2, поэтому

$$A_F \geq \frac{1}{2} \sqrt{2} \sqrt{(2^n - 1)(2^{n-1} - 1)}.$$

Для $n \geq 3$ имеем $2^n - 1 \geq 2^{n-1}$ и $2^{n-1} - 1 \geq 2^{n-2}$. Следовательно,

$$(2^n - 1)(2^{n-1} - 1) \geq 2^{n-1} \cdot 2^{n-2} = 2^{2n-3},$$

и тогда

$$A_F \geq \frac{1}{2} \sqrt{2} \sqrt{2^{2n-3}} = \frac{1}{2} \sqrt{2} \cdot 2^{(2n-3)/2} = 2^{n-2}.$$

В частности $A_F \geq 2^{(n-1)/2}$ для всех $n \geq 3$ (так как $n - 2 \geq (n - 1)/2$ при $n \geq 3$).

Из связи $\delta_\pi = A_F/2^{n-1}$ получаем

$$\delta_\pi \geq \frac{2^{(n-1)/2}}{2^{n-1}} = 2^{-\frac{n-1}{2}}.$$

Таким образом, получаем оценку для нижней границы линейной характеристики.

$$\delta_\pi \geq 2^{-\frac{n-1}{2}}$$

Алгебраическая степень

Пусть

$$\pi(x) = (f_1(x), f_2(x), \dots, f_n(x)),$$

где $f_i: V_n \rightarrow \mathbb{F}_2$ – координатные функции. Для $\alpha \in V_n \setminus \{0\}$ рассмотрим линейную комбинацию координатных функций

$$\alpha \cdot \pi(x) = \bigoplus_{i=1}^n \alpha_i f_i(x).$$

Аналогично определим $\alpha \cdot \pi^{-1}(x)$. Алгебраическая степень подстановки определяется как

$$\lambda_\pi = \min_{\alpha \in V_n \setminus \{0\}} \{\deg(\alpha \cdot \pi(x)), \deg(\alpha \cdot \pi^{-1}(x))\}$$

где $\deg(f)$ – степень многочлена Жегалкина функции f . Большое значение λ_π затрудняет построение низкостепенных алгебраических моделей и повышает устойчивость к алгебраическим атакам [11].

Нижняя граница:

$$\lambda_\pi \geq 1.$$

Доказательство: покажем, что для любой $\alpha \neq 0$ функция $\alpha \cdot \pi(x)$ не может быть константой. Если бы $\alpha \cdot \pi(x) \equiv c \in \mathbb{F}_2$, то множество образов $\pi(V_n)$ лежало бы целиком в аффинной гиперплоскости $\{y \in V_n: \alpha \cdot y = c\}$, имеющей мощность 2^{n-1} . Но π – биекция, следовательно $|\pi(V_n)| = |V_n| = 2^n$, что противоречит оценке мощности гиперплоскости. Значит $\alpha \cdot \pi(x)$ не является константой, а потому $\deg(\alpha \cdot \pi(x)) \geq 1$. Аналогично для $\alpha \cdot \pi^{-1}(x)$. Следовательно,

$$\min \{\deg(\alpha \cdot \pi), \deg(\alpha \cdot \pi^{-1})\} \geq 1$$

для любого $\alpha \neq 0$, откуда нижняя граница $\lambda_\pi \geq 1$.

Верхняя граница:

$$\lambda_{\pi} \leq n - 1.$$

Доказательство: пусть $\pi: V_n \rightarrow V_n$ - подстановка, $\pi(x) = (f_1(x), \dots, f_n(x))$, где $f_i: V_n \rightarrow \mathbb{F}_2$. Для $\alpha \in V_n \setminus \{0\}$ введём булевы функции

$$g_{\alpha}(y) = \alpha \cdot y = \bigoplus_{i=1}^n \alpha_i y_i$$

$$g_{\alpha} \circ \pi(x) = \alpha \cdot \pi(x) = \bigoplus_{i=1}^n \alpha_i f_i(x)$$

$$g_{\alpha}(x) \circ \pi^{-1}(x) = \alpha \cdot \pi^{-1}(x)$$

Пусть $h: V_n \rightarrow \mathbb{F}_2$ - произвольная булева функция и пусть её полином Жегалкина записан в индексной форме как

$$h(x) = a \oplus \left(\bigoplus_{1 \leq i_1 \leq \dots \leq i_k \leq n} a_{i_1, \dots, i_k} \cdot x_{i_1} \cdot \dots \cdot x_{i_k} \right),$$

$$a, a_{i_1, \dots, i_k} \in \{0, 1\}, \quad k \in [1, n]$$

Для фиксированного набора индексов $i_1 < \dots < i_k$ сумма значений монома $x_{i_1} \cdots x_{i_k}$ по всем $x \in V_n$ равна 2^{n-k} . Следовательно, вклад всех мономов с $k < n$ в сумму $\sum_{x \in V_n} h(x)$ чётен, а вклад монома при $k = n$ (то есть при $i_1 = 1, \dots, i_n = n$) равен 1. Отсюда

$$\sum_{x \in V_n} h(x) \equiv a_{1, 2, \dots, n} \pmod{2}.$$

Линейная форма $g_{\alpha}(y) = \alpha \cdot y$ при ненулевом α принимает значение 1 ровно на 2^{n-1} входах y . Так как π - биекция, композиция $g_{\alpha} \circ \pi$ принимает значение 1 также ровно на 2^{n-1} входах x , аналогично $g_{\alpha} \circ \pi^{-1}$ принимает значение 1 на 2^{n-1} входах. То есть для любого ненулевого α функции $\alpha \cdot \pi$ и $\alpha \cdot \pi^{-1}$

сбалансированы, и их суммы значений по V_n чётны. По предыдущему наблюдению это означает, что коэффициент полного монома $a_{1,2,\dots,n}$ в полиноме Жегалкина каждой из этих функций равен нулю. Следовательно, ни одна из функций $\alpha \cdot \pi$ и $\alpha \cdot \pi^{-1}$ не содержит монома порядка n , то есть их степени не превосходят $n - 1$.

Поскольку α был произвольным ненулевым вектором, по определению λ_π имеем

$$\lambda_\pi \leq n - 1.$$

Таким образом, ограничения алгебраической степени подстановки таковы:

$$1 \leq \lambda_\pi \leq n - 1$$

Выбранная ARX-конструкция

В рамках настоящей работы рассматривается криптографическая конструкция, реализующая подстановку

$$\pi: V_8 \rightarrow V_8,$$

основанную на принципах ARX (Addition–Rotation–XOR). Преобразование действует над 8-битными векторами, которые на каждом раунде интерпретируются как пара 4-битных блоков:

$$x \in V_8 \Rightarrow x = (X_L, X_R), \quad X_L, X_R \in V_4.$$

Одна итерация ARX-преобразования описывается далее.

Описание одной итерации ARX-преобразования

Сначала производится циклический сдвиг правого блока:

$$R_1 = X_R^{(i-1)} \lll r_i,$$

где $\lll r_i$ - циклический сдвиг влево на $r_i \in [0,3]$ бит. При $r_i = 0$ операция является тождественной.

Далее следует сложение по модулю 2:

$$Z^{(i)} = X_L^{(i-1)} \boxplus R_1,$$

где $\boxplus$ – побитовое сложение по модулю 2.

После чего происходит второй циклический сдвиг:

$$R_2 = Z^{(i)} \lll s_i,$$

где $s_i \in [0,3]$.

За ним следует обновление правого блока через XOR:

$$X_R^{(i)} = R_2 \oplus X_R^{(i-1)}.$$

И обновление левого блока:

$$X_L^{(i)} = Z^{(i)}.$$

Что выливается в итоговый выходной вектор:

$$x^{(i)} = (X_L^{(i)}, X_R^{(i)}) \in V_8.$$

Далее приведено визуальное представление 1 итерации преобразования

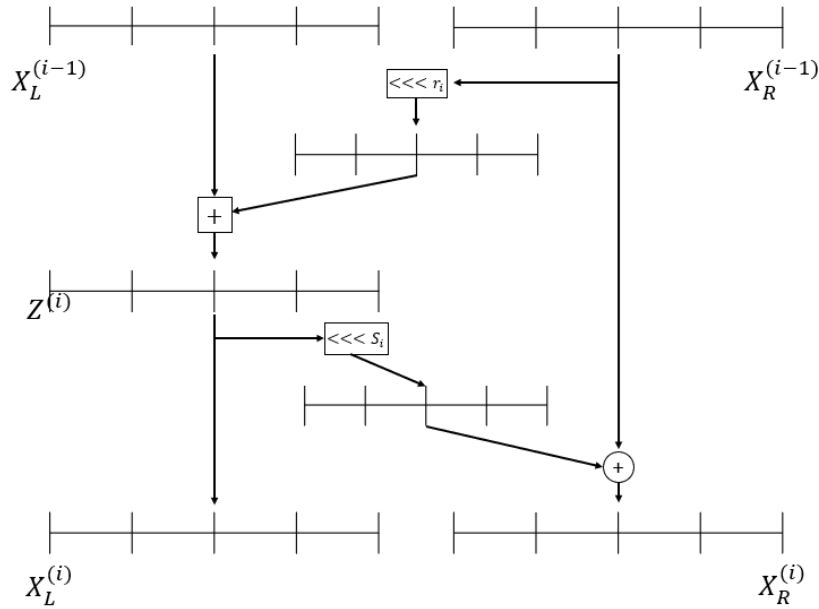


Рисунок 1. Структура одной итерации ARX-конструкции над 8-битным вектором

Композиция итераций

Раундовая ARX-подстановка $\pi_T: V_8 \rightarrow V_8$ с параметром числа итераций $T \geq 1$ определяется как композиция:

$$\pi_T = F^{(T)} \circ F^{(T-1)} \circ \dots \circ F^{(1)}.$$

Для любого входного вектора $x \in V_8$:

$$\pi_T(x) = x^{(T)} = F^{(T)}(F^{(T-1)}(\dots F^{(1)}(x) \dots)).$$

Параметризация

Каждая конкретная подстановка π_T задаётся набором параметров:

$$(r_1, s_1), (r_2, s_2), \dots, (r_T, s_T) \in [0,3]^{2T}.$$

Таким образом, общее число возможных конфигураций равно:

$$N_T = 16^T.$$

Далее представлена таблица количества конфигураций ARX-подстановки в зависимости от количества итераций:

T	$N_T = 16^T$
1	16
2	256
3	4 096
4	65 536
5	1 048 576
6	16 777 216
7	268 435 456
8	4 294 967 296
9	68 719 476 736

Таблица 1. Количество возможных конфигураций ARX-подстановки при различных значениях числа итераций

Методы вычисления

Количество возможных конфигураций в зависимости от количества итераций растет экспоненциально, из-за чего вычисление характеристик может оказаться трудной задачей. По этой причине необходимо понимать важность выбора методов вычисления, так как одни методы могут привести к очень продолжительным расчетам, а другие могут существенно уменьшить количество времени вычислений. Существует два принципиально отличающихся друг от друга метода, о которых будет рассказано далее.

Начнем с грубого метода вычисления. Грубый метод представляет собой прямой, исчерпывающий перебор входных значений и явное накопление всех необходимых статистик для фиксированной конфигурации параметров $(r_1, s_1), \dots, (r_T, s_T)$. Методологически он прост и математически прозрачен, поэтапное описание данного метода приведено далее.

Грубые методы вычисления характеристик

В качестве единого предварительного этапа следует построить эталонный массив образов Y , где для каждого входа $x \in V_n$ вычислено и сохранено $Y[x] = \pi(x)$. Этот массив служит источником данных для всех последующих процедур и исключает повторные вычисления $\pi(x)$.

Грубый метод вычисления r_π предусматривает следующие шаги:

1. Инициализировать матрицу счётчиков DDT размером $2^n \times 2^n$ нулями (строки - входные разности α , столбцы - выходные разности β).
2. Для каждой входной разности α ($\alpha \neq 0$) и для каждой β требуется найти x такой, что $\beta = Y[x \oplus \alpha] \oplus Y[x]$ и увеличить соответствующий счётчик $DDT[\alpha, \beta]$ на единицу.
3. После полного перебора для каждой пары (α, β) получить частоту перехода $DDT[\alpha, \beta]$, после чего нормировать делением на 2^n для получения вероятности.
4. Найти максимальное значение счётчика (или максимальную нормированную вероятность) по всем $\alpha \neq 0, \beta$. Это и есть r_π .

Далее представлена возможная реализация алгоритма на языке Python:

```
N = len(Y)
DDT = [ [0]*N for _ in range(N) ]

for alpha in range(1, N):
    for beta in range(N):
        cnt = 0
    for x in range(N):
        if (Y[x ^ alpha] ^ Y[x]) == beta:
            cnt += 1
        DDT[alpha][beta] = cnt

Delta = 0
for alpha in range(1, N):
    for beta in range(N):
        if DDT[alpha][beta] > Delta:
            Delta = DDT[alpha][beta]

p_pi = Delta / N
```

Алгоритм даёт точное значение p_π , но требует $O(2^{3n})$ элементарных операций при данной реализации, где n – длина входного вектора, то есть 8. С точки зрения памяти, алгоритм требует $O(2^{2n})$ для хранения полной матрицы DDT, а также $O(2^n)$ для массива образов Y .

При переборе всех конфигураций число операций умножается на число конфигураций $N_T = 16^T$, поэтому суммарная стоимость растёт как

$$16^T \cdot O(2^{3n}).$$

Для вычисления λ_π дополнительно требуется массив обратной подстановки Y^{-1} , где для каждого $y \in V_n$ хранится $Y^{-1}[y] = x$ такое, что $Y[x] = y$.

Грубый метод вычисления λ_π предусматривает следующие шаги:

1. Для каждой ненулевой маски $\alpha \in V_n$ построить булеву функцию $f_\alpha(x) = \alpha \cdot \pi(x) = \alpha \cdot Y[x]$ (скалярное произведение битов), представив её в виде таблицы истинности длины 2^n .
2. Для каждой такой таблицы истинности вычислить коэффициенты АНФ по определению:

$$\alpha[m] = \bigoplus_{x:(x \& m)=m} f_\alpha(x)$$

данное выражение можно интерпретировать как коэффициент при мономе с маской m равен XOR всех значений функции на надмножествах m .

3. По полученным коэффициентам определить степень $\deg(f_\alpha)$ как максимальную мощность подмножества переменных ($\text{popcount}(m)$), для которых коэффициент ненулевой.
4. Аналогично, для той же маски α построить таблицу истинности $g_\alpha(x) = \alpha \cdot \pi^{-1}(x) = \alpha \cdot Y^{-1}[x]$, получить её АНФ и степень $\deg(g_\alpha)$.
5. Для каждой α вычислить $\deg_\alpha = \max \{\deg(f_\alpha), \deg(g_\alpha)\}$.
6. Взять $\lambda_\pi = \min_{\alpha \neq 0} \deg_\alpha$.

Далее представлена возможная реализация алгоритма на языке Python:

```
def build_inverse(Y):
    N = len(Y)
    Yinv = [0] * N
    for x in range(N):
        Yinv[Y[x]] = x
    return Yinv

def truth_table_alpha(Y, alpha):
    return [((alpha & Y[x]).bit_count()) & 1 for x in range(len(Y))]

def truth_table_alpha_inv(Yinv, alpha):
    return [((alpha & Yinv[x]).bit_count()) & 1 for x in range(len(Yinv))]

def anf_coeffs(truth):
    N = len(truth)
    coeffs = [0] * N
    for m in range(N):
        acc = 0
        for x in range(N):
            if (x & m) == m:
                acc ^= truth[x]
            coeffs[m] = acc
    return coeffs

def degree_from_anf(coeffs):
    deg = 0
    for mask, c in enumerate(coeffs):
        if c:
            bits = mask.bit_count()
            if bits > deg:
                deg = bits
    return deg

def compute_lambda_pi(Y):
    N = len(Y)
    Yinv = build_inverse(Y)
    lambda_val = None

    for alpha in range(1, N):

        t_f = truth_table_alpha(Y, alpha)
        a_f = anf_coeffs(t_f)
        deg_f = degree_from_anf(a_f)

        t_g = truth_table_alpha_inv(Yinv, alpha)
        a_g = anf_coeffs(t_g)
        deg_g = degree_from_anf(a_g)

        deg_alpha = deg_f if deg_f >= deg_g else deg_g
        if (lambda_val is None) or (deg_alpha < lambda_val):
            lambda_val = deg_alpha

    return lambda_val
```

Для каждой из $2^n - 1$ масок α строится таблица истинности $O(2^n)$, производится вычисление коэффициентов АНФ, что дает $O(n2^n)$ операций, то

есть суммарно $O(n2^n)$ на α , что в итоге дает для каждой ненулевой маски, которых всего $2^n - 1$, общее количество $O((2^n - 1) \cdot 2^{2n}) \approx O(2^{3n})$ операций, где n – длина входного вектора, то есть 8.

С точки зрения памяти, требуется хранить Y , Y^{-1} и временные массивы длины 2^n для таблицы истинности и коэффициентов АНФ, при последовательной обработке масок дополнительная память не растёт. Таким образом, необходимая память $O(2^n)$. В итоге, алгоритм даёт точное λ_π при умеренной памяти $O(2^n)$, но время на одну конфигурацию растёт как $O(2^{3n})$, что при увеличении количества итераций может оказать значительное влияние на время вычисления.

При переборе всех конфигураций число операций умножается на число конфигураций $N_T = 16^T$, поэтому суммарная трудоемкость растёт как

$$16^T \cdot O(2^{3n}).$$

Далее рассмотрим грубый метод вычисления δ_π . Алгоритм предусматривает следующие шаги:

1. Для всех пар масок $(u, v) \in V_n \times V_n$ последовательно вычислять сумму

$$S(u, v) = \sum_{x \in V_n} (-1)^{u \cdot x \oplus v \cdot Y[x]},$$

где $u \cdot x$ и $v \cdot Y[x]$ – скалярные произведения по модулю 2.

2. Для каждой пары (u, v) запоминать абсолютное значение $|S(u, v)|$.
3. После перебора всех пар взять максимальное значение $\max_{u,v} |S(u, v)|$ и нормировать его делением на 2^n . Полученная величина

$$\delta_\pi = \frac{1}{2^n} \max_{u,v} |S(u, v)|$$

- есть искомая линейная характеристика постановки.

Далее представлена возможная реализация алгоритма на языке Python

```
N = len(Y)
max_abs_S = 0

for u in range(N):
    for v in range(N):
        S = 0
        for x in range(N):
            bit = ((u & x).bit_count() ^ (v & Y[x]).bit_count()) & 1
            if bit == 0:
                S += 1
            else:
                S -= 1
        if abs(S) > max_abs_S:
            max_abs_S = abs(S)

delta_pi = max_abs_S / N
```

Далее будут проанализированы сложность и затраты по памяти для алгоритма. Три вложенных перебора по u , v и x дают $2^n \cdot 2^n \cdot 2^n$ суммирований, то есть общая сложность $O(2^{3n})$ операций. С точки зрения памяти, $O(2^n)$ при хранении только массива образов Y и текущего максимума. В итоге при T итерациях число операций умножается на 16^T , поэтому общее количество операций увеличивается до $16^T \cdot O(2^{3n})$.

Грубый метод для δ_π прост и даёт точное значение, но его временная сложность $O(2^{3n})$ делает его непрактичным при массовом переборе конфигураций. Грубый метод возможно использовать для валидации или на малых T , но для исследования он не годится, из-за чего было принято решение использовать более эффективные методы, о которых подробно будет рассказано далее.

Эффективные методы вычисления характеристик

Заранее зафиксируем, что под эффективностью методов подразумевается минимизация количества операций алгоритма без снижения точности получаемого результата – искомой характеристики. Так как количество операций напрямую связано с временем вычислений, в то время как требования по памяти находятся в рамках допустимых значений.

Рассмотрим эффективный метод вычисления разностной характеристики подстановки.

1. Для каждой ненулевой входной разности $\alpha \in V_n \setminus \{0\}$ - инициализируем счётчики для выходных разностей β .
2. Для всех $x \in V_n$ вычисляем:

$$\beta = \pi(x) \oplus \pi(x \oplus \alpha).$$

Увеличиваем счётчик для соответствующей β .

3. После полного прохода по всем x для данной α определяем:

$$M(\alpha) = \max_{\beta} DDT[\alpha, \beta].$$

4. Определяем глобальный максимум по всем α :

$$M_{\max} = \max_{\alpha \neq 0} M(\alpha).$$

5. Вычисляем характеристику:

$$p_{\pi} = \frac{M_{\max}}{2^n}.$$

Сложность

- Суммарное количество операций $O(2^{2n})$.
- Память - $O(2^n)$

Уменьшение количества операций достигнуто за счет исключения тройного перебора, так как конкретный индекс β вычисляется для каждого x , метод предполагает алгоритмическую оптимизацию, нежели математическую, но

она дает асимптотическое улучшение по количеству операций, что и было необходимо.

Рассмотрим эффективный метод вычисления алгебраической степени подстановки.

Строим обратный массив

$$Y^{-1}[y] = x \text{ для всех } y \in V_n \text{ таких, что } Y[x] = y.$$

Для каждой ненулевой маски $\alpha \in V_n \setminus \{0\}$:

1. Строим таблицу истинности для f_α

$$f_\alpha(x) = (\alpha, Y[x]) = \bigoplus \sum_{i=1}^n \alpha_i \cdot y_i, x \in V_n.$$

Получаем вектор t_f длины N .

2. Строим таблицу истинности для g_α

$$g_\alpha(x) = (\alpha, Y^{-1}[x]), x \in V_n.$$

Получаем вектор t_g длины N .

Далее необходимо получить коэффициенты АНФ.

Для каждого из векторов t_f и t_g применяем конструкцию K . Для блоков v_1, v_2 одинаковой длины, K определяется так:

$$K(v_1, v_2) = v_1 \parallel (v_1 \oplus v_2).$$

Таким образом, итеративно сворачиваем таблицу истинности, сначала по парам битов, затем по блокам длины 2, затем 4 и так до длины 2^{n-1} . Финальный вектор длины N есть коэффициенты АНФ.

Цель конструкции K - преобразовать таблицу истинности функции $h: V_n \rightarrow \{0,1\}$ в вектор коэффициентов её АНФ. Конструкция действует блочно и эквивалентна subset-Möbius-преобразованию.

Далее необходимо определить степени. Для полученного вектора коэффициентов $a[m]$:

- Каждому индексу m соответствует маска переменных.
- Степень монома = $\text{popcount}(m)$.
- Степень функции:

$$\deg(h) = \max \{ \text{popcount}(m) \mid a[m] = 1 \}.$$

Таким образом:

$$\begin{aligned} \deg(f_\alpha) &= \max \{ \text{popcount}(m) \mid a_f[m] = 1 \}, \deg(g_\alpha) \\ &= \max \{ \text{popcount}(m) \mid a_g[m] = 1 \}. \end{aligned}$$

Агрегируем по маске:

$$\deg_\alpha = \max \{ \deg(f_\alpha), \deg(g_\alpha) \}.$$

Вычисляем итоговое значение – искомую характеристику:

$$\lambda_\pi = \min_{\alpha \in V_n \setminus \{0\}} \deg_\alpha.$$

Сложность

- Суммарное количество операций $O(n2^{2n})$.
- Память: $O(2^n)$ при последовательной обработке.

Рассмотрим небольшой пример для $n = 3$, то есть для 3-битной подстановки. Выберем маску $\alpha = 101_2$, пусть задана таблица подстановки π , где для каждого x заданы $y = \pi(x)$ побитово как (y_3, y_2, y_1) . Таблица имеет вид:

$x =$	x_3	x_2	x_1	y_3	y_2	y_1	$y = \pi(x)$
0	0	0	0	0	0	1	1
1	0	0	1	1	0	0	4
2	0	1	0	0	0	0	0

3	0	1	1	1	0	1	5
4	1	0	0	0	1	0	2
5	1	0	1	1	1	1	7
6	1	1	0	0	1	1	3
7	1	1	1	1	1	0	6

Порядок столбцов (y_3, y_2, y_1) соответствует старшему, среднему и младшему битам соответственно. Получается, подстановка имеет вид:

$$\pi(x) = \begin{pmatrix} 0 & 1 & 2 & 3 & 4 & 5 & 6 & 7 \\ 1 & 4 & 0 & 6 & 2 & 7 & 3 & 6 \end{pmatrix}, \text{ где } \pi[0] = \begin{pmatrix} 0 \\ 1 \end{pmatrix} = 1$$

Маска $\alpha = (1, 0, 1)$, это означает, что в скалярном XOR-умножении берутся биты y_3 и y_1 , а y_2 игнорируется. Таким образом

$$f_\alpha(x) = \alpha \cdot \pi(x) = y_1 \oplus y_3$$

Далее строим таблицу истинности для f_α

	x_3	x_2	x_1	$f_\alpha = (\alpha, \pi(x))$
0	0	0	0	1
1	0	0	1	1
2	0	1	0	0
3	0	1	1	0
4	1	0	0	0
5	1	0	1	0
6	1	1	0	1
7	1	1	1	1

Получаем вектор таблицы истинности $t_f = 11000011_2$, теперь к полученному вектору применяется выше определенная конструкция К. Для блоков v_1, v_2 одинаковой длины определенная как

$$K(v_1, v_2) = v_1 \parallel (v_1 \oplus v_2),$$

где $\parallel$ –конкатенация, $\oplus$ –XOR.

Шаг 1. По парам битов (блоки длины 1)

Разбиваем t_f на пары и применяем К к каждой паре:

- Пара $(t_0, t_1) = (1, 1)$:
 - $t_0 = [1], t_1 = [1]$
 - $z_0 = [1 \parallel (t_0 \text{ XOR } t_1)] = [1, 0]$
- Пара $(t_2, t_3) = (0, 0)$:
 - $t_2 = [0], t_3 = [0]$
 - $z_1 = [0 \parallel (t_2 \text{ XOR } t_3)] = [0, 0]$
- Пара $(t_4, t_5) = (0, 0)$:
 - $z_2 = [0 \parallel (t_4 \text{ XOR } t_5)] = [0, 0]$
- Пара $(t_6, t_7) = (1, 1)$:
 - $z_3 = [1 \parallel (t_6 \text{ XOR } t_7)] = [1, 0]$

Получаем четыре блока длины 2: $z_0 = [1, 0], z_1 = [0, 0], z_2 = [0, 0], z_3 = [1, 0]$.

Шаг 2. По блокам длины 2

Применяем К к парам блоков длины 2:

- $K(z_0, z_1) = z_0 \parallel (z_0 \text{ XOR } z_1)$
 - $z_0 \text{ XOR } z_1 = [1 \wedge 0, 0 \wedge 0] = [1, 0]$
 - $c_0 = [1, 0, 1, 0]$
- $K(z_2, z_3) = z_2 \parallel (z_2 \text{ XOR } z_3)$
 - $z_2 \text{ XOR } z_3 = [0 \wedge 1, 0 \wedge 0] = [1, 0]$

$$\circ c_1 = [0,0,1,0]$$

Получаем два блока длины 4: $c_0 = [1,0,1,0], c_1 = [0,0,1,0]$.

Шаг 3. По блокам длины 4

Финальный шаг: $K(c_0, c_1) = c_0 \parallel (c_0 \text{ XOR } c_1)$

- $c_0 \text{ XOR } c_1 = [1^{\wedge}0, 0^{\wedge}0, 1^{\wedge}1, 0^{\wedge}0] = [1, 0, 0, 0]$
- $k = [1, 0, 1, 0, 1, 0, 0, 0]$

На финальном шаге получаем вектор k длиной 8 бит, это и есть вектор коэффициентов АНФ $a_f = (1, 0, 1, 0, 1, 0, 0, 0)$.

Данные шаги могут быть представлены в качестве таблицы, которая наглядно показывает действие конструкции:

0	1	2	3	4	5	6	7
1	1	0	0	0	0	1	1
1	0	0	0	0	0	1	0
1	0	1	0	0	0	1	0
1	0	1	0	1	0	0	0
Свободный член	x_1	Коэффициент при x_2	x_1x_2	Коэффициент при x_3	x_1x_3	x_2x_3	$x_1x_2x_3$

Получены индексы с единицей в АНФ – 0, 2, 4. Таким образом:

$$f_{\alpha}(x_1, x_2, x_3) = x_3 \oplus x_2 \oplus 1$$

Максимальная степень среди мономов – 1, следовательно

$$\deg(f_{\alpha}) = 1$$

Данный метод дает выигрыш по количеству операций, что сказывается на эффективности, следующим образом:

$$\frac{O(2^{3n})}{O(n2^{2n})} = \frac{2^n}{n} = \frac{256}{8} = 32$$

Получается, выигрыш в численном выражении – 32, что более наглядно демонстрируется в следующей таблице:

Количество итераций T	16^T	Грубый метод (количество операций)	Предложенный метод (количество операций)
1	16	268,435,456	8,388,608
2	256	4,294,967,296	134,217,728
3	4,096	68,719,476,736	2,147,483,648
4	65,536	1,099,511,627,776	34,359,738,368
5	1,048,576	17,592,186,044,416	549,755,813,888

Теперь рассмотрим эффективный алгоритм вычисления δ_π

Определение функции g^β и таблицы истинности

- Фиксируем маску $\beta \neq 0$ и вводим булеву функцию

$$(\beta, x) = g^\beta(x) = \bigoplus_{i=1}^n \beta_i \cdot y_i, \quad y = \pi(x), x \in V_n$$

- Строим вектор t_{g^β} длины N со значениями $t_{g^\beta}[x] = g^\beta(x)$ для всех $x \in V_n$.

Далее необходимо получить коэффициенты Фурье. Для вектора t_{g^β} применяем блочную конструкцию K' итеративно по уровням. K' - для двух блоков v_1, v_2 одинаковой длины имеет вид:

$$K'(v_1, v_2) = (v_1 + v_2) || (v_1 - v_2),$$

где «+» и «-» - покомпонентные целочисленные операции.

Далее производим итеративное сворачивание (аналогично конструкции K) последовательно по уровням:

- пары значений (блоки длины 1),
- блоки длины 2,
- блоки длины 4,
- ... до длины 2^{n-1} .

После n уровней получаем целочисленные коэффициенты Фурье $C_\alpha^{g^\beta}$ (вектор длины N), где индекс α трактуется как маска в стандартном порядке индексов.

Далее необходимо нормировать коэффициенты и вычислить $\delta_{\alpha,\beta}^\pi$

- Коэффициенты $C_\alpha^{g^\beta}$ нормируются делением на 2^{n-1} .
- Вычисляем корреляции

$$\delta_{\alpha,\beta}^\pi = - \frac{C_\alpha^{g^\beta}}{2^{n-1}}.$$

Значение $\delta_{\alpha,\beta}^\pi$ лежит в диапазоне $[-1,1]$.

Перебор масок β и чтение δ по α

- Повторяем шаги 1–3 для всех $\beta \in V_n \setminus \{0\}$
- Из полученного вектора коэффициентов для данного β вычислить все $\delta_{\alpha,\beta}^\pi$ по формуле выше.

Остается только вычислить итоговое значение искомой характеристики

- Вычисляем характеристику по известной формуле

$$\delta_\pi = \max_{\alpha \in V_n \setminus \{0\}, \beta \in V_n \setminus \{0\}} |\delta_{\alpha,\beta}^\pi|$$

где

$$\delta_{\alpha,\beta}^\pi = - \frac{C_\alpha^{g^\beta}}{2^{n-1}}, \quad g^\beta(x) = (\beta, \pi(x)).$$

Сложность

- Суммарное количество операций $O(n2^{2n})$.
- Память – $O(2^n)$ при последовательной обработке.

Данный метод дает выигрыш по количеству операций, что сказывается на эффективности, следующим образом:

$$\frac{O(2^{3n})}{O(n2^{2n})} = \frac{2^n}{n} = \frac{256}{8} = 32$$

Реализация вычислений характеристик

Как отмечалось ранее, число возможных конфигураций 8-битных подстановок растёт экспоненциально: при параметризации пространства кандидатов оно достигает величины $N = 16^T$. Уже при малых значениях T объём поиска становится настолько большим, что ручное вычисление криптографических характеристик оказывается невозможным. Это обстоятельство делает необходимой полную автоматизацию процесса вычислений.

Для решения задачи была выбрана реализация алгоритмов на языке C, поскольку он сочетает высокую скорость выполнения с низкоуровневым контролем над памятью и вычислительными ресурсами.

Реализация эталонного кода на Си

В качестве отправной точки, было принято решение реализовать вышеописанные методы с минимальными изменениями на языке Си, данный код послужит эталоном при дальнейших оптимизациях и переносе. Его задача – быть гарантом корректности вычислений, этот этап очень важен, так как малейшая ошибка на данном этапе может стать причиной лавинного эффекта в будущем, когда вычисления при больших T будут занимать значительное время. Эталонная реализация не преследует высокую производительность, главное получить прочный каркас, на основе которого будут проходить дальнейшие эксперименты. Такое решение упростит отладку и настройку дальнейших реализаций, так как будет возможно проверять корректность работы.

Для начала требовалось реализовать вспомогательные части, как, например, циклический сдвиг, нахождение степени АНФ, один раунд ARX-конструкции, а также раундовую вариацию. Далее представлен фрагмент реализации вспомогательных частей кода, которые будут использоваться в вычислениях характеристик.

```
static inline uint8_t cls4(uint8_t v, uint8_t shift) {
    shift &= 3; // mod 4
    uint8_t x = (v << shift) & 0x0F;
    uint8_t y = (v >> (4 - shift)) & 0x0F;
    return (x | y) & 0x0F;
}

static inline uint8_t A_single(uint8_t X, uint8_t r, uint8_t S) {
    uint8_t X1 = (X >> 4) & 0x0F;
    uint8_t X2 = X & 0x0F;

    uint8_t Z = (X1 + cls4(X2, r)) & 0x0F;
    uint8_t Y2 = (X2 ^ cls4(Z, S)) & 0x0F;

    return (uint8_t)((Z << 4) | Y2);
}

static uint8_t A_rounds(uint8_t X, const uint8_t *r, const uint8_t *S,
uint8_t T) {
    uint8_t Y = X;
    for (uint8_t i = 0; i < T; ++i) {
        Y = A_single(Y, r[i], S[i]);
    }
}
```

```

return Y;
}

```

Далее было необходимо реализовать функции вычисления необходимых характеристик, так как все необходимое уже есть. Первой была написана функция вычисления разностной характеристики:

```

static float compute_p_pi(const uint8_t Y[256]) {
    int counter[256][256] = {0};
    float max_prob = 0.0f;

    for (int x = 0; x < 256; x++) {
        for (int a = 1; a < 256; a++) {
            int x_prime = x ^ a;
            int eps = Y[x] ^ Y[x_prime];
            if (eps == 0) continue;
            counter[a][eps]++;
        }
    }

    for (int a = 1; a < 256; a++) {
        int total = 0;
        for (int eps = 1; eps < 256; eps++) total += counter[a][eps];
        if (total == 0) continue;

        for (int eps = 1; eps < 256; eps++) {
            float prob = (float)counter[a][eps] / (float)total;
            if (prob > max_prob) max_prob = prob;
        }
    }
    return max_prob;
}

```

Представленный код демонстрирует реализацию предложенного эффективного метода вычисления разностной характеристики p_π . Далее была написана функция для вычисления алгебраической степени λ_π :

```

static int compute_lambda(uint8_t *Y) {
    int lambda = 8;
    uint8_t Yinv[256];

    for (int x = 0; x < 256; x++) {
        Yinv[Y[x]] = x;
    }

    for (int a = 1; a < 256; a++) {
        uint8_t f[256], g[256];

        for (int x = 0; x < 256; x++) {
            f[x] = __builtin_parity(a & Y[x]);
        }
        for (int x = 0; x < 256; x++) {
            g[x] = __builtin_parity(a & Yinv[x]);
        }

        int deg_f = anf_degree(f);
    }
}

```

```

    int deg_g    = anf_degree(g);
    int max_deg  = (deg_f > deg_g ? deg_f : deg_g);

    if (max_deg < lambda) lambda = max_deg;
}
return lambda;
}

```

Функция явно демонстрирует представленный метод. Последняя функция – функция вычисления линейной характеристики δ_π :

```

static float compute_delta(const uint8_t Y[256]) {
    const float norm = 1.0f / 256.0f;
    float max_abs_delta = 0.0f;

    for (int beta = 1; beta < 256; beta++) {
        int f[256];

        for (int x = 0; x < 256; x++) {
            f[x] = __builtin_parity(beta & Y[x]) ? -1 : 1;
        }

        fwt_int_256(f);

        for (int alpha = 1; alpha < 256; alpha++) {
            float delta_alpha_beta = -(float)f[alpha] * norm;
            float abs_delta = delta_alpha_beta < 0 ? -delta_alpha_beta :
delta_alpha_beta;
            if (abs_delta > max_abs_delta) max_abs_delta = abs_delta;
        }
        return max_abs_delta;
    }
}

```

Основная функция (main) состоит из ввода параметров, вызова функций и вывода в консоль для удобства. Стоит отметить, что в функции так же предусмотрена проверка биективности на тот случай, когда полученное преобразование не является подстановкой. Полный код эталонной реализации представлен в [Листинг 1](#).

Оптимизация вычислений с использованием OpenMP и SIMD

Эталонная реализация позволяет вычислить точные характеристики, но на $T \geq 4$ время вычислений становилось слишком большим, что не допустимо в рамках исследования, так как требовалось проверить подстановки вплоть до $T = 9$. В таком случае возникает необходимость оптимизации программы. В текущем исполнении при отсутствии ключей компилятора, которые производят оптимизацию и векторизацию, программа использует одно ядро центрального процессора и один поток. В рамках оптимизационных мер будет проведена векторизация с помощью SIMD (Single Instruction, Multiple Data), а также распараллеливание с помощью библиотеки Open MP (Open Multi-Processing) [12]. Таким образом, с помощью векторизации получится ускорить внутренние вычисления в каждом потоке, а также распределить задачи между ядрами, нагрузив полностью центральный процессор. Скорость работы программы будет ограничена лишь возможностями центрального процессора, а точнее количеством ядер, их частотой и количеством потоков.

Для эффективной векторизации были добавлены предвычисленные значения некоторых функций (lookup-таблицы), таблицы выровнены и компактны, что позволяет компилятору эффективнее векторизовать циклы. Данные lookup-таблицы достаточно малы (порядка нескольких килобайт), что позволяет помещаться им в L1/L2 кэш процессора, что кратно ускоряет доступ к ним, так как нет необходимости обращаться к основной памяти. Далее представлен фрагмент кода, демонстрирующий изменения:

```
static uint8_t cls_table[16][4];
static uint8_t A_single_cache[16][16][4][4];
static bool tables_initialized = false;

void init_tables() {
    if (tables_initialized) return;

    for (uint8_t val = 0; val < 16; ++val) {
        for (uint8_t shift = 0; shift < 4; ++shift) {
```

```

        cls_table[val][shift] = ((val << shift) | ((val << shift) >> 4))
        & 0x0F;
    }
}

for (uint8_t X1 = 0; X1 < 16; ++X1) {
    for (uint8_t X2 = 0; X2 < 16; ++X2) {
        for (uint8_t r = 0; r < 4; ++r) {
            for (uint8_t S = 0; S < 4; ++S) {
                uint8_t Z = (X1 + cls_table[X2][r]) & 0x0F;
                uint8_t Y2 = (X2 ^ cls_table[Z][S]) & 0x0F;
                A_single_cache[X1][X2][r][S] = (Z << 4) | Y2;
            }
        }
    }
}
tables_initialized = true;
}

```

Таким образом, вместо вычислений циклического сдвига и одного раунда преобразования берется готовый результат из таблицы. Из-за чего функция, возвращающая результат одного раунда преобразования, стала выглядеть следующим образом:

```

static inline uint8_t A_single_opt(const uint8_t X, const uint8_t r, const
uint8_t S) {
    return A_single_cache[(X >> 4) & 0x0F][X & 0x0F][r][S];
}

```

Обновленная функция вычисления разностной характеристики имеет вид:

```

static float compute_differential_prob_opt(const uint8_t *sbox) {
    alignas(32) uint16_t P[256][256] = {0};
    float max_prob = 0.0f;

    #pragma omp parallel for reduction(max:max_prob)
    for (int a = 1; a < 256; ++a) {
        uint16_t local_P[256] = {0};
        for (int x = 0; x < 256; ++x) {
            int x_xor_a = x ^ a;
            local_P[sbox[x] ^ sbox[x_xor_a]]++;
        }

        for (int b = 0; b < 256; ++b) {
            float prob = (float)local_P[b] / 256.0f;
            if (prob > max_prob) max_prob = prob;
        }
    }
    return max_prob;
}

```

Обновленная функция вычисления алгебраической степени λ_π не претерпела весомых алгоритмических изменений, кроме аналогичного внедрения параллелизма, в следствие чего нет необходимости демонстрации.

В функции вычисления линейной характеристики δ_π было произведено небольшое количество изменений в целях оптимизации. Метод один и тот же – спектральный через преобразование Уолш-Адамар (FWT), но алгоритм переделан для эффективной параллельности. Далее представлена обновленная функция вычисления линейной характеристики:

```
static float compute_delta_opt(const uint8_t sbox[256]) {
    alignas(32) int f[256];
    float max_delta = 0.0f;

    #pragma omp parallel for reduction(max:max_delta)
    for (int beta = 1; beta < 256; beta++) {
        for (int x = 0; x < 256; x++) {
            f[x] = (parity(beta & sbox[x]) == 0) ? 1 : -1;
        }

        int len = 128;
        while (len > 0) {
            for (int start = 0; start < 256; start += 2 * len) {
                for (int i = start; i < start + len; i++) {
                    int a = f[i];
                    int b = f[i + len];
                    f[i] = a + b;
                    f[i + len] = a - b;
                }
            }
            len >>= 1;
        }

        for (int alpha = 1; alpha < 256; alpha++) {
            float bias = fabsf((float)f[alpha]) / 256.0f;
            if (bias > max_delta) {
                max_delta = bias;
            }
        }
    }
    return max_delta;
}
```

Помимо изменений в основных частях программы, был внесен целый ряд изменений:

- Добавлена новая структура данных Combination с полями для параметров и характеристик, чтобы удобно хранить и сравнивать результаты, она имеет вид:

```
typedef struct {
    float prob;
    int lambda_pi;
    float linear_bias;
    uint8_t r[MAX_T];
    uint8_t S[MAX_T];
} Combination;
```

- Реализован механизм отбора наилучших комбинаций по нескольким критериям ($p_\pi, \lambda_\pi, \delta_\pi$), включая проверку доминирования и обновление с помощью Парето-фронта.
- Оптимизирована проверка на биективность с помощью битовой маски, что быстрее и компактнее.
- Для корректной работы SIMD использовалось выравнивание памяти для массивов.
- Добавлены новые элементы CLI (Command Line Interface) для удобства.

Полный код оптимизированной программы представлен в [Листинге 2](#).

Помимо оптимизации кода было принято решение воспользоваться ключами компилятора, которые оптимизируют программу, но не влияют на точность, то есть не упрощают вычисления с плавающей точкой, так как точность является существенным требованием в данном исследовании. Далее приведен список всех используемых ключей при компиляции программы:

- -O3 – агрессивная оптимизация, разворачивающая циклы, которая может автоматически векторизовать части программы.
- -march-native – использовать все инструкции процессора (AVX2, AVX-512, SSE).
- -fopenmp – включает поддержку директив `#pragma omp`.

Существует так же очень агрессивная оптимизация со стороны компилятора, которая включается с помощью ключа -Ofast, но в таком случае нельзя гарантировать точность, вследствие возможного упрощения вычислений с плавающей точкой.

Сравнение производительности эталонного кода и оптимизированной программы

Для демонстрации внедренных ранее оптимизаций было проведено сравнение программ по времени вычислений. Стоит отметить, что ожидается увидеть уменьшение времени работы не более чем в 4 раза, так как процессор, использующийся для вычислений обладал лишь 4 ядрами и 8 потоками. Ускорение более в 4 раза маловероятно, так как операционная система и иные процессы на компьютере так же используют центральный процессор в своих целях. Сравнение проводилось для $T \in [1,6]$. Для наглядного сравнение результаты представлены в виде таблицы:

T (Количество итераций)	Эталонный код	Оптимизированная программа	Коэффициент ускорения
1	≈ 0	≈ 0	-
2	≈ 0	≈ 0	-
3	3	1	3
4	67	26	2,58
5	830 $\approx$ 13,8 минут	281 $\approx$ 4,68 минут	2,96
6	13251 $\approx$ 3,68 часов	4500 $\approx$ 1,25 часов	2,94
7		71290 $\approx$ 19,8 часов	-

Таблица 2. Сравнение времени работы эталонного кода и оптимизированной программы

Таблица наглядно демонстрирует прирост скорости в 2,5-3 раза, что является неплохим результатом. Данный коэффициент, как было сказано ранее, зависит от аппаратной составляющей, на процессорах с большим количеством ядер с аналогичной частотой коэффициент ускорения будет больше. Данная оптимизация, позволила сократить время вычисления для $T = 7$ с потенциальных 60 часов до 20, более наглядным становится ускорение на 8-

ми итерациях, в таком случае эталонному коду потребовалось бы 40 суток (если считать, что времени потребуется ровно в 16 раз больше, чем для $T = 7$: $\frac{60 \cdot 16}{24} = 40$), а оптимизированной программе около 13 суток.

Даже большее ускорение, например, на более мощном процессоре не гарантирует вычисление характеристик для $T = [8, 9]$ за допустимое время, так как если для $T = 8$ потребуется 13 суток, что уже сложно назвать допустимым временем, то для $T = 9$ время увеличивается уже до 208 суток, что точно указывает на необходимость изменения подхода.

Подходов может быть несколько, зависит от того, под каким углом рассматривается задача. Было выявлено два явных пути, где накладываются ограничения, далее о каждом по порядку.

Один из подходов подразумевает уменьшение количества вычислений, что явным образом влияет на время работы программы. Идея состоит в том, чтобы вместо полного перебора всех конфигураций брать случайные выборки заданного размера и вычислять характеристики только для них, отбрасывая оставшиеся. Предложенный метод похож на метод Монте-Карло по своей сути. Предположим, что ресурсы позволяют проводить вычисления лишь для 7 итераций, полный перебор остальных невозможен. В таком случае можно брать случайную выборку на дальнейших итерациях, тогда размер максимальный допустимый размер выборки будет составлять $\frac{1}{16}$ всех конфигураций на 8 итерациях и $\frac{1}{256}$ всех конфигураций на 9 итерациях. На практике интеграция данного метода является достаточно простой, но неэффективной. После нескольких экспериментов стало понятно, что подход неудачный, из-за чего был отброшен.

Поскольку реализация имеет аппаратную зависимость, одним из существенных ограничений выступают характеристики вычислительной платформы. В таком случае возможно перенести программу на иную систему,

где аппаратная составляющая будет достаточно мощной, чтобы провести вычисления за допустимый промежуток времени. Так как ограничения накладывают в основном процессор, следует экстраполировать полученную статистику на лучшие серверные решения. В таком случае рассмотрим 3 варианта:

- AMD EPYC 9754 (Zen 4c, 128C/256T)
- AMD Threadripper PRO 7995WX (Zen 4, 96C/192T)
- Intel Xeon Platinum (Sapphire Rapids, 56–64C)

Построим грубую оценку, которой будет достаточно для принятия дальнейших решений. Для начала следует определить базовую идею:

$$\text{время} \approx \frac{\text{работа}}{\text{производительность}},$$

где производительность является произведением следующих параметров, имеющих численное выражение (объединение закона Амдала [13], закона Густавсона-Бариса [14], RooflineModel [15]):

- C — количество физических ядер
- f — эффективная тактовая частота
- I — IPC (инструкций на 1 такт)
- W — ширина SIMD

Существуют и другие метрики, такие как эффективность памяти, эффективность параллелизации и т.д., но они были опущены, так как не оказывают существенного влияния, точнее почти не отличаются от 1. Тогда, грубая формула ускорения вычислений на сервере будет иметь вид:

$$k = \frac{C_s}{C} \cdot \frac{f_s \cdot I_s}{f \cdot I} \cdot \frac{W_s}{W}$$

Проведем вычисление потенциального ускорения на самом мощном из представленных процессоре AMD EPYC 9754, для него необходимые характеристики известны:

$$k = 32 \cdot 1.25 \cdot 1.6 = 64$$

В таком случае, вычисления вместо 208 суток заняли бы 3,25 суток. На данном процессоре возможно получить максимальное потенциально возможно ускорение, то есть на иных серверных процессорах коэффициент будет ниже. Такой подход может сравнимо ускорить вычисления, но даже на лучшем процессоре все равно потребуется более 3 суток. Данный подход возможен в случае неограниченных ресурсов или же доступности необходимых компонентов. Данное исследование ограничено по аппаратной базе, из-за чего такой подход не подходит.

Третий подход подразумевает уход от ограничений процессора в сторону графических ускорителей. Так как в программе производится множество параллельных вычислений, то следует использовать устройства оптимизированные под задачи такого рода, а именно графические процессоры или же видеокарты. Для реализации третьего подхода требуется перенести вычисления на графический процессор, что является отдельной задачей, вследствие специфики платформы. Среди имеющихся аппаратных ресурсов присутствует видеокарта Nvidia RTX 5070 Ti, поэтому программу потребуется перенести на CUDA, о чем подробно будет рассказано далее.

Реализация вычислений на CUDA

CUDA (Compute Unified Device Architecture) — это параллельная вычислительная платформа и модель программирования от NVIDIA, позволяющая использовать графические процессоры для выполнения общих вычислений [16]. API представляет из себя некоторое расширение языка C++. Далее будет рассказано о специфике работы с CUDA.

Особенности разработки на CUDA

Стоит начать с того, что программный код разделяется на две принципиально разные части. Первой частью является хост-код, который выполняется на центральном процессоре, данный код управляет запуском, вторая же часть содержит GPU-код, который реализует ядра CUDA, которые исполняются параллельно множеством потоков. Таким образом, CPU иницирует выполнение, а GPU выполняет основную нагрузку. Передача данных между хостом и устройством осуществляется через шину PCIe. Существует иерархия потоков, которая представлена:

- Grid (сетка) - совокупность всех блоков, запущенных для выполнения ядра.
- Block (блок) - группа потоков, которые могут взаимодействовать через общую память (sharedmemory).
- Thread (поток) - минимальная единица исполнения, обрабатывающая отдельный элемент данных.

Каждый поток имеет уникальные индексы (threadIdx, blockIdx, blockDim, gridDim), что позволяет обращаться к конкретным элементам массива или матрицы. В CUDA также значительно отличается модель памяти от привычной:

- Глобальная память доступна всем потокам, но обладает высокой латентностью, оптимизация требует коалесцированного доступа (соседние потоки обращаются к последовательным адресам).
- Sharedmemory - быстрая память, общая для всех потоков блока, используется для обмена данными и уменьшения обращений к глобальной памяти.
- Регистры - локальные для каждого потока, самые быстрые, но ограниченные по объёму.
- Константная и текстурная память – это специализированные области, оптимизированные для определённых типов доступа.

Эффективность CUDA-программы напрямую зависит от грамотного использования этих уровней памяти.

Для разработки на CUDA компанией Nvidia поставляется комплект CUDA Toolkit, в состав которого входит собственный компилятор командной строки nvcc. Компилятор nvcc создан на основе открытого компилятора Open64 и предназначен для трансляции хост-кода и GPU-кода в объектные файлы. Основные задачи компилятора nvcc:

- принимает исходники с CUDA-расширениями (.cu)
- разделяет код на хост-часть и GPU-часть
- транслирует GPU-код в PTX (Parallel Thread Execution- промежуточный ассемблер GPU) или в машинный код SASS для конкретных архитектур;
- вызывает стандартный C/C++ компилятор (gcc, cl.exe и др.) для компиляции хост-кода;
- объединяет всё в единый исполняемый файл или библиотеку.

NVCC - ключевой компонент CUDA Toolkit. Он связывает:

- CUDA Runtime API (функции cudaMalloc, cudaMemcpy);

- CUDA Driver API (низкоуровневый интерфейс к драйверу GPU);
- библиотеки (cuBLAS, cuFFT, cuDNN и др.);
- инструменты профилирования (nvprof, Nsight).

Помимо всего вышеперечисленного существуют и другие отличия и особенности, но в их разборе нет необходимости. Отдельно стоит отметить, что в качестве ключа компилятора следует явно указывать архитектуру, на которой будет работать данная программа. Архитектура зависит от типа графического процессора (десктопный, серверный) и поколения. В рамках данного исследования используется архитектура sm_89. Далее будет описана подготовка и перенос программы на CUDA.

Подготовка и перенос кода на платформу CUDA

Перенос существующей программы с языка Си на CUDA требует систематической подготовки исходного кода. Основная цель – выделить те части алгоритма, которые могут быть эффективно распараллелены на графическом процессоре, и адаптировать их к модели исполнения CUDA.

Для начала требуется определить критические циклы и участки кода, где выполняется массовая обработка данных, например, перебор масок, свертки в FWT. Далее выделить функции, которые можно вынести в отдельные CUDA-ядра. В общем, проверяются зависимости между итерациями, так как независимые хорошо подходят для параллельного запуска потоков.

Следующий шаг – разделение программы на хост-код и GPU-код. Здесь критерием являются задачи отдельной части кода, если код управляющий, то его следует поместить в хост часть, код же осуществляющий конкретные вычисления вынести в GPU часть. Таким образом, получается логическое разделение кода, одна часть останется в практически неизменном виде, иная будет адаптирована под GPU.

Далее необходимо провести работу с памятью. Все массивы и структуры данных, используемые в вычислениях, должны быть перенесены в память GPU. Для этого в хост-коде добавляются вызовы `cudaMalloc`, `cudaMemcpy` и `cudaFree`. Важно заранее определить, какие данные будут храниться в глобальной памяти (доступной всем потокам), а какие можно разместить в `sharedmemory` (общей для блока) для ускорения доступа. В исходном Си-коде массивы должны быть организованы так, чтобы доступ к ним был последовательным и коалесцированным – это повышает эффективность работы GPU.

Теперь нужно подготовить функции и циклы. Последовательные циклы в Си-коде преобразуются в параллельные вызовы потоков CUDA. Каждый поток получает свой индекс (`threadIdx`, `blockIdx`) и обрабатывает соответствующий элемент данных. Функции, выполняющие арифметические операции над массивами (например, блочная конструкция K' для FWHT), переписываются как CUDA-ядра. Важно минимизировать использование глобальных переменных и обеспечить независимость вычислений каждого потока.

Также на этапе переноса добавляются специфические для CUDA макросы и директивы, определяющие, например, область выполнения функции. Таким образом, управляющие функции, то есть функции, запускающие вычисления остаются практически без изменений, а функции, реализующие вычисления переписываются под графический процессор, например, функция вычисления разностной характеристики принимает вид:

```
__device__ uint16_t compute_ddt_max_dev(const uint8_t sbox[256]) {
    uint16_t global_max = 0, row[256];
    for (int a = 1; a < 256; ++a) {
        #pragma unroll 1
        for (int i = 0; i < 256; ++i) row[i] = 0;
        #pragma unroll 1
        for (int x = 0; x < 256; ++x) {
            uint8_t b = sbox[x] ^ sbox[x ^ a];
            row[b]++;
        }
        #pragma unroll 1
        for (int b = 0; b < 256; ++b)
```

```
        if (row[b] > global_max) global_max = row[b];  
    }  
    return global_max;  
}
```

Результатом переноса становится полностью рабочая версия программы на языке CUDA, полный код представлен в [Листинге 3](#). Далее программа компилируется с помощью nvcc, результатом чего становится исполняемый файл. Далее проводится тестирование на малых T, результаты сравниваются с результатами эталонной программы.

Сравнение производительности различных вариантов программ

Для наглядного сравнения стоит учитывать все версии программы, написанные до этого момента. Далее представлена таблица с результатами сравнения скорости работы эталонного кода, эталонного кода, оптимизированного с помощью SIMDи OpenMP, а также программы написанной на CUDA. Последний столбец содержит коэффициент ускорения, формально - частное значения программы на CUDAи значения эталонного кода.

Т (Количество итераций)	Эталонный код	Программа, написанная на CUDA	Коэффициент ускорения
1	≈ 0	≈ 0	-
2	≈ 0	≈ 0	-
3	3	≈ 0	-
4	67	≈ 0	-
5	830 $\approx 13,8$ минут	3.22	257,7
6	13251 $\approx 3,68$ часов	52	254,8
7	-	836 $\approx 13,9$ минут	-
8	-	13467 $\approx 3,74$ часов	-
9	-	216822 $\approx 60,22$ часов	-

Таблица 3. Сравнение времени работы эталонного кода и программы на CUDA

В данном случае, коэффициент ускорения значительно выше, чем в случае с оптимизированным кодом, выполняющемся на процессоре, ≈ 250 против 3.

Валидация полученных характеристик

Проверка полученных результатов является важной частью работы, так как именно на этапе проверки выясняется, не проделана ли работа впустую. В данном случае требуется проверить значения характеристик, так как малейшие ошибки и даже небольшие отклонения от истинных значений могут привести к ложным выводам, что не является допустимым в рамках работы.

Ранее отмечалось, что для валидации результатов можно использовать эталонный код, например, дописав возможность вычисления характеристик для конкретной параметризации. Такой подход возможен, но также возможно, что на этапе написания эталонного кода были допущены ошибки. В таком случае проверка производится не может.

В качестве альтернативного варианта был выбран язык программирования Python, так как для него существует модуль Sage, в котором содержатся функции необходимые для вычисления характеристик. Для проверки был написан скрипт, который вычисляет характеристики выбранной параметризации и сравнивает с полученными от программы на CUDA с точностью 9 знаков после запятой.

Проверять каждую параметризацию с помощью Python не является хорошим решением по тем же причинам, почему программа для вычисления характеристик была сначала модифицирована, а в конечном итоге переписана. Python не подходит для данной задачи, так как он является достаточно медленным, из-за чего проверять каждую параметризацию не представляется возможным. Было принято компромиссное решение – проверять лучшие параметризации по композиции признаков и некоторые случайно выбранные. Такое решение позволяет гарантировать корректность характеристик за допустимый промежуток времени.

Скрипт на языке Python, использующий для вычислений характеристик библиотеку Sage, выполняющий валидацию конкретных параметризаций находится в [Листинге 4](#).

Глава 2. Экспериментальная часть и результаты

Данная глава полностью посвящена описанию проведенных экспериментов, а также обсуждению полученных результатов. До обсуждения экспериментальной части стоит зафиксировать аппаратные ресурсы, так как программные решения очень от них зависят. Эксперименты проводились на персональном компьютере со следующей видеокартой:

- Графический процессор - MSI GeForce RTX 5070 Ti VENTUS 3X OC, 16 Гб видеопамяти, частота работы видеочипа – 2497 МГц, 8960 CUDA ядер

Представленная конфигурация будет использоваться на протяжении всей экспериментальной части. В качестве операционной системы будет использоваться Windows 11 x64, так как на данной операционной системе разработка на CUDA оказалась наиболее эффективной.

Описание экспериментов

Экспериментальная часть делилась на несколько этапов. Первым этапом служило вычисление характеристик для малых $T \in [1,6]$ с помощью эталонной программы. Время вычислений фиксировалось для анализа, после чего проводилась валидация результатов с помощью Python скрипта. Вторым этапом заключался в вычислении характеристик для $T \in [1,7]$ с дальнейшей валидацией и замерами времени.

Заключительным этапом экспериментальной части стало вычисление характеристик для $T \in [1,9]$ программой на языке CUDA. Еще на начальных T было принято решение фиксировать не только параметризации, но и гистограммы. Такое решение позволяет анализировать распределение значений характеристик по всем параметризациям. Поэтому была написана функция, собирающая данные и записывающая в файл в формате:

p_π	λ_π	δ_π	Count	Percent
---------	---------------	--------------	-------	---------

Таким образом стало возможным отслеживать количество параметризаций с конкретным набором параметров, а также их процент среди всех параметризаций. Благодаря гистограммам можно узнать насколько редко встречаются лучшие параметризации по композиции характеристик. Гистограммы могут быть использованы также для выявления закономерностей, что может помочь сделать обоснованные предположения о значениях характеристик для $T > 9$.

Сравнение реализаций

Собранные во время экспериментальной части данные позволяют провести сравнение времени работы различных реализаций программы. Далее в таблице будет представлена общая таблица, содержащая время работы для различных вариантов программы:

T (Количество итераций)	Эталонный код	Оптимизированный эталонный код	Программа на CUDA
1	≈ 0	≈ 0	≈ 0
2	≈ 0	≈ 0	≈ 0
3	3	1	≈ 0
4	67	26	≈ 0
5	830 $\approx$ 13,8 минут	281 $\approx$ 4,68 минут	3.22
6	13251 $\approx$ 3,68 часов	4500 $\approx$ 1,25 часов	52
7		71290 $\approx$ 19,8 часов	836 $\approx$ 13,9 минут
8			13467 $\approx$ 3,74 часов
9			216822 $\approx$ 60,22 часов

Таблица 4. Сравнение времени работы различных вариаций программы

Таблица наглядно демонстрирует колоссальное отличие в скорости работы различных вариантов программы. Легко заметить, что необходимость переноса программы на платформу CUDA была оправданной, зная коэффициент ускорения, указанный в Таблице 3, можно посчитать примерное время, которое бы заняли расчёты для $T = 9$ с помощью эталонной программы:

$$60,22 \times 250 = 15055 \text{ часов} \approx 627 \text{ суток} \approx 21 \text{ месяц}$$

Результаты вычислений

В результате вычислений для каждой $T \in [1,9]$ были получены два файла, первый содержит параметризацию и вычисленные характеристики в формате:

$r[r_1, \dots, r_T]$	$S[S_1, \dots, S_T]$	p_π	λ_π	δ_π
----------------------	----------------------	---------	---------------	--------------

Файл разделен на несколько секций:

- Парето-оптимальные комбинации – лучшие параметризации по композиции параметров
- Параметризации с минимальной разностной характеристикой
- Параметризации с минимальной линейной характеристикой

В каждом разделе содержатся по 20 явно заданных параметризаций по неубыванию показателя секции (композиция, разностная характеристика, линейная характеристика). Такое решение позволяет находить не только параметризации с лучшей композицией, но и находить минимальные разностную и линейную характеристики для каждого T . Параметризации с наибольшей алгебраической степенью не выделялись, так как данная характеристика на больших T чаще всего являлась максимальной у большинства параметризаций.

Второй файл, как и было написано в описании экспериментальной части, содержит гистограмму.

p_π	λ_π	δ_π	Count	Percent
---------	---------------	--------------	-------	---------

Рассмотрим в качестве примеров фрагменты файлов, полученных после вычислений для $T = 5$. Фрагменты представлены в виде таблиц для удобства восприятия, полные файлы находятся на GitHub по ссылке

<https://github.com/ShishulinDmitry/diploma>. Для начала рассмотрим файл, содержащий лучшие параметризации. Общий формат файла:

$r[r_1, \dots, r_T]$	$S[S_1, \dots, S_T]$	p_π	λ_π	δ_π
----------------------	----------------------	---------	---------------	--------------

Фрагмент секции Парето-оптимальных комбинаций:

[2 1 3 2 1]	[1 0 3 0 0]	0.0468750	5	0.2500000
[3 2 2 3 1]	[3 0 3 0 0]	0.0390625	6	0.2812500
[0 2 0 3 0]	[0 1 3 0 0]	0.0468750	6	0.2656250
...				

Фрагмент секции параметризаций с минимальной разностной характеристикой:

[2 3 2 3 0]	[3 0 3 3 3]	0.0390625	6	0.2968750
[0 3 2 3 0]	[3 0 3 3 3]	0.0390625	6	0.3125000
[0 3 3 0 2]	[2 3 2 3 3]	0.0390625	6	0.2812500
...				

Фрагмент секции параметризаций с минимальной линейной характеристикой:

[3 2 0 3 2]	[0 3 2 3 3]	0.0468750	5	0.2500000
[3 2 0 3 0]	[0 3 2 3 3]	0.0468750	5	0.2500000
[2 1 3 3 2]	[1 0 2 3 3]	0.0703125	6	0.2500000
...				

Фрагмент файла, содержащего гистограмму:

p_π	λ_π	δ_π	Count	Percent
0.0390625	6	0.2812500	16	0.0015259%
0.0390625	6	0.2968750	32	0.0030518%
0.0390625	6	0.3125000	48	0.0045776%
0.0468750	3	0.5000000	16	0.0015259%
0.0468750	4	0.2812500	16	0.0015259%
0.0468750	4	0.2968750	48	0.0045776%
0.0468750	4	0.3125000	192	0.0183105%
0.0468750	4	0.3281250	64	0.0061035%
0.0468750	4	0.3750000	112	0.0106812%
0.0468750	4	0.5000000	64	0.0061035%
0.0468750	5	0.2500000	32	0.0030518%
0.0468750	5	0.2656250	48	0.0045776%
0.0468750	5	0.2812500	96	0.0091553%
0.0468750	5	0.2968750	192	0.0183105%
0.0468750	5	0.3125000	528	0.0503540%
0.0468750	5	0.3281250	208	0.0198364%

Теперь стоит уделить внимание лучшим результатам, которые будут собраны в одну таблицу, что упростит визуальный анализ и сравнение.

T	p_π	λ_π	δ_π	Count	%
1	1	1	1	16	100%
2	0.5	1	1	96	37.5%
3	0.25	3	0.5	176	4.2968750%
4	$0.0781250\frac{5}{64}$	4	$0.3281250\frac{21}{64}$	16	0.0244141%
	0.0703125	5	0.5000000	64	0.0976562%

	$\frac{9}{128}$				
5	0.0468750 $\frac{3}{64}$	5	0.2500000 $\frac{1}{4}$	32	0.0030518%
	0.0390625 $\frac{5}{128}$	6	0.2812500 $\frac{9}{32}$	16	0.0015259%
6	0.0390625 $\frac{5}{128}$	7	0.2500000 $\frac{1}{4}$	880	0.0052452%
	0.0390625 $\frac{5}{128}$	6	0.2343750 $\frac{15}{64}$	48	0.0002861%
7	0.0390625 $\frac{5}{128}$	7	0.2343750 $\frac{15}{64}$	2848	0.0010610%
	0.0312500 $\frac{1}{32}$	7	0.2500000 $\frac{1}{4}$	16	0.0000060%
8	0.0312500 $\frac{1}{32}$	7	0.2343750 $\frac{15}{64}$	64	0.0000015%
	0.0390625 $\frac{5}{128}$	7	0.2187500 $\frac{7}{32}$	16	0.0000004%
9	0.0312500 $\frac{1}{32}$	7	0.2343750 $\frac{15}{64}$	1200	0,0000017%
	0.0390625 $\frac{5}{128}$	7	0.2187500 $\frac{7}{32}$	64	0,000000093%

Таблица 5. Характеристики лучших параметризаций для всех рассмотренных T

Каждому набору значений характеристик (строке вида $p_\pi, \lambda_\pi, \delta_\pi$) соответствует конкретная параметризация, то есть каждая строка является отдельной подстановкой. Легко заметить, что с увеличением количества итераций «Count» количество лучших параметризаций увеличивается, а процент параметризациям с лучшими характеристиками уменьшается. Это является наглядным подтверждением, что решение о выборочной проверке было верно отвергнуто, так как даже в случае скопления всех подстановок в определенном месте, вероятность попасть в этот кластер довольно-таки мала, не говоря о случае, когда они распределены равномерно.

Далее будут представлены лучшие значения каждой характеристики для каждой отдельной T :

T	p_π	λ_π	δ_π
1	1	1	1
2	0.5	1	1
3	0.25	3	0.5
4	0.0625000 $\frac{1}{16}$	5	0.3125000 $\frac{1}{32}$
5	0.0390625 $\frac{5}{128}$	6	0.2500000 $\frac{1}{4}$
6	0.0390625 $\frac{5}{128}$	7	0.2343750 $\frac{15}{64}$
7	0.0312500 $\frac{1}{32}$	7	0.2343750 $\frac{15}{64}$
8	0.0312500 $\frac{1}{32}$	7	0.2187500 $\frac{7}{32}$

9	0.0312500 $\frac{1}{32}$	7	0.2187500 $\frac{7}{32}$
---	-----------------------------	---	-----------------------------

Таблица 6. Лучшие отдельные характеристики для каждой рассмотренной T

Представленные данные указывают на неплохие характеристики получаемых подстановок, также заметно плато, на котором значения не меняются, но судя по данным, эти плато держатся несколько итераций, после чего снова наступает улучшение.

Чтобы оценить насколько хорошими являются полученные характеристики, требуется провести сравнение с известными подстановками, которые являются частями стандартов. Следующий раздел будет посвящен сравнению полученных характеристик с характеристиками S-Box-подстановки шифра AES и подстановки ГОСТ «Кузнечик».

Сравнение с AES и ГОСТ «Кузнечик»

Сначала следует зафиксировать, что участвовать в сравнении будут конкретные подстановки, то есть параметризации. Лучшие параметризации представлены в Таблице 5. Выделим лучшие параметризации по композиции характеристик среди всех представленных:

T	p_{π}	λ_{π}	δ_{π}
9	$\frac{1}{32} = \frac{4}{128}$	7	$\frac{15}{64}$
	$\frac{5}{128}$	7	$\frac{7}{32} = \frac{14}{64}$

Таблица 7. Характеристики лучших параметризаций по композиции параметров для $T=9$

Для удобства будем пользоваться дробным представлением, а также приведем к одному знаменателю значения каждой характеристики. Теперь рассмотрим характеристики подстановок, выбранных для сравнения:

	p_{π}	λ_{π}	δ_{π}
AES	$\frac{1}{64} = \frac{2}{128}$	7	$\frac{16}{128} = \frac{8}{64}$
ГОСТ «Кузнечик»	$\frac{1}{32} = \frac{4}{128}$	7	$\frac{7}{32} = \frac{14}{64}$

Таблица 8. Характеристики эталонных подстановок

Легко заметить, что полученные параметризации по своим характеристикам очень близки к характеристикам подстановки ГОСТ «Кузнечик»:

- Первая параметризация обладает той же разностной характеристикой, но значение линейной характеристики на $\frac{1}{64}$ больше (в 1.25 раза), чем у «Кузнечика».

- Вторая параметризация наоборот имеет такую же линейную характеристику, но разностная характеристика больше на $\frac{1}{128}$ (в 1.07 раз больше).

Таким образом, уже на 9 итерациях находятся подстановки с очень близкими к подстановке «Кузнечика» характеристиками.

Далее нужно сравнить с S-Box-подстановкой шифра AES. Заранее стоит отметить, что подстановка AES обладает очень хорошей композицией параметров, поэтому ее можно использовать в качестве эталона.

- Разностная характеристика первой параметризации ровно в 2 раза больше, чем разностная характеристика подстановки AES ($\frac{4}{128}$ против $\frac{2}{128}$), в случае линейной характеристики ситуация схожа – линейная характеристика первой параметризации в 1.875 раз больше, чем у подстановки AES ($\frac{15}{64}$ против $\frac{8}{64}$)
- Разностная характеристика второй параметризации в 2.5 раза больше, чем у подстановки AES ($\frac{5}{128}$ против $\frac{2}{128}$), а линейная - в 1.75 раз больше ($\frac{14}{64}$ против $\frac{8}{64}$).

Для наглядности стоит объединить все характеристики, которые сравнивались, в общую таблицу, что позволит построчно анализировать, как близки полученные характеристики к эталонным:

	1	2	AES	ГОСТ «Кузнечик»
p_{π}	$\frac{4}{128}$	$\frac{5}{128}$	$\frac{2}{128}$	$\frac{4}{128}$
λ_{π}	7	7	7	7
δ_{π}	$\frac{15}{64}$	$\frac{14}{64}$	$\frac{8}{64}$	$\frac{14}{64}$

Таблица 9. Сравнение полученных характеристик с эталонными

Сравнение показало, что были достигнуты хорошие результаты близкие к значениям характеристик ГОСТ «Кузнечик» и в то же время ≈ 2 раза хуже, чем у S-Box-подстановки AES, что нельзя трактовать как неудачу, так как целью является исследование возможностей, а не получение конкретного результата.

Отдельно стоит отметить редкость лучших параметризаций. Как видно из Таблицы 5, количество параметризаций с лучшими для конкретного T характеристиками достаточно мало, в случае $T = 9$ они составляют лишь $1,7 \times 10^{-6} \%$ и $9,3 \times 10^{-8} \%$ от количества всех параметризаций соответственно. Эта редкость обусловлена экспоненциальным увеличением количества вариаций параметризаций.

Рассмотрим возможность использования подстановок из первой и второй параметризаций в качестве нелинейного преобразования в алгоритме блочного шифрования «Кузнечик», а также оценим стойкость указанного алгоритма относительно разностного и линейного методов при использовании как оригинальной подстановки «Кузнечика», так и подстановок из параметризаций, полученных в рамках данной работы.

В работе [17] в зависимости от числа итераций t определяется показатель рассеивания линейной среды алгоритма блочного шифрования относительно разностного $\theta_p(t)$ (линейного $\theta_\Delta(t)$) метода как минимально возможное (по всем системам локальных согласованных разностных (линейных вероятностных) соотношений) число нелинейных функциональных элементов (т.н. активных подстановок), необходимых для построения согласованной системы локальных разностных (линейных) соотношений на число итераций, равное t . Таким образом, зная точное значение (или оценку снизу) для $\theta_p(t)$ и $\theta_\Delta(t)$, в соответствии с, например [18], для блочного шифра (у которого на каждой итерации все подстановки одинаковы с заданными характеристиками p_π и δ_π) с длиной блока n бит для любых $\alpha, \beta \in V_n \setminus \{0\}$ для преобразования

F_t (где F_t – преобразование зашифрования t итерациями заданного блочного шифра), имеем следующие неравенства $P_{\alpha,\beta}^{F_t} \leq (p_\pi)^{\theta_p(t)}$ и $|\delta_{\alpha,\beta}^{F_t}| \leq (|\delta_\pi|)^{\theta_\pi(t)}$. В таком случае в соответствии с [18] и учитывая, что при применении разностного (линейного) метода к алгоритму блочного шифрования с использованием конкретного разностного (линейного) соотношения (α, β) для преобразования F_t количество минимально необходимого материала составляет не менее $\frac{1}{P_{\alpha,\beta}^{F_t}} \left(\frac{1}{(\delta_{\alpha,\beta}^{F_t})^2} \right)$, имеем, что минимально возможное количество итераций t блочного шифра, на котором используемое разностное (линейное) соотношение нельзя будет статистически различить вычисляется из условий $p_\pi^{\theta_p(t)} \leq 2^{-n}$ и $(\delta_\pi)^{2 \cdot \theta_\pi(t)} \leq 2^{-n}$.

Далее, для линейных преобразований, используемых в алгоритмах блочного шифрования, важной криптографической характеристикой является коэффициент рассеивания линейного преобразования ($\rho(L)$ – определение см., например, в [16]), с использованием которого и выражения (25) работы [16] для алгоритма «Кузнечик» можно вычислить $\theta_p(t)$ и $\theta_\pi(t)$. Напомним, что линейное преобразование L , используемое в алгоритме блочного шифрования «Кузнечик» является максимально рассеивающим (т.е. значение $\rho(L)$ максимально при заданном разбиении $n = m \cdot n' = 16 \cdot 8 = 128$ и равно 17). Также в соответствии с теоремой 2 работы [20] для алгоритма «Кузнечик» выполняется $\theta_p(t) = \theta_\pi(t) = \theta(t)$. В итоге, в таблице 10 приведены значения $\theta(t)$ для алгоритма «Кузнечик», полученные в соответствии с выражением (25) работы [19].

t	1	2	3	4	5	6	7	8	9
-----	---	---	---	---	---	---	---	---	---

θ	1	17	18	34	35	51	52	68	69
----------	---	----	----	----	----	----	----	----	----

Таблица 10. Зависимость показателя рассеивания линейной среды алгоритма «Кузнечик» от числа итераций

Так как на каждой итерации алгоритма «Кузнечик» используется одно и то же преобразование $S: V_{128} \rightarrow V_{128}$, заключающееся в параллельном применении 16 подстановок $\pi: V_8 \rightarrow V_8$, то согласно изложенному выше количество итераций t , на которое максимально существует разностное (линейное) соотношение, различаемое на всем доступном материале 2^{128} , определяется из неравенств $p_{\pi}^{\theta(t)} \leq 2^{-128}$ и $(\delta_{\pi})^{2\theta(t)} \leq 2^{-128}$.

Напомним, что для подстановки $\pi: V_8 \rightarrow V_8$, используемой в алгоритме «Кузнечик», $p_{\pi} = \frac{4}{128} = \frac{1}{32}$ и $\delta_{\pi} = \frac{7}{32}$. В таком случае, минимальное число итераций, для которых в алгоритме «Кузнечик» любое разностное соотношение и линейный статаналог не смогут быть различаемыми на всем материале в 2^{128} , известных пар открытого-шифрованного текста определяется из следующих систем неравенств.

$$\begin{cases} p_{\pi}^{\theta(t)} \leq 2^{-128} \\ (\delta_{\pi})^{2\theta(t)} \leq 2^{-128} \end{cases} \Leftrightarrow \begin{cases} 5 \cdot \theta(t) \geq 128 \\ (5 - \log_2 7) \cdot 2\theta(t) \geq 128 \end{cases} \Leftrightarrow \begin{cases} \theta(t) \geq 25,6 \\ \theta(t) \geq 29,19 \end{cases}. \quad (1)$$

Что дает нам минимальное количество итераций, на котором любое разностное (линейное) соотношение не смогут быть различаемыми на всем материале в 2^{128} , равным 4.

Рассмотрим неравенства $p_{\pi}^{\theta(t)} \leq 2^{-128}$ и $(\delta_{\pi})^{2\theta(t)} \leq 2^{-128}$ для подстановок из второй параметризации, имеющих характеристики $p_{\pi} = \frac{5}{128} = \frac{1,25}{32}$ и $\delta_{\pi} = \frac{7}{32}$. Система неравенств (1) для них примет вид:

$$\begin{cases} p_{\pi}^{\theta(t)} \leq 2^{-128} \\ (\delta_{\pi})^{2\theta(t)} \leq 2^{-128} \end{cases} \Leftrightarrow \begin{cases} 4,68 \cdot \theta(t) \geq 128 \\ (5 - \log_2 7) \cdot 2\theta(t) \geq 128 \end{cases} \Leftrightarrow \begin{cases} \theta(t) \geq 27,35 \\ \theta(t) \geq 29,19 \end{cases}.$$

Что также дает нам минимальное количество итераций, на котором любое разностное (линейное) соотношение не смогут быть различаемыми на всем материале в 2^{128} , равным 4. Таким образом, использование в алгоритме блочного шифрования «Кузнечик» подстановок из второй параметризации со значениями $p_\pi = \frac{5}{128}$ и $\delta_\pi = \frac{7}{32}$ не ухудшает его стойкость относительно разностного и линейного методов.

Далее рассмотрим неравенства $p_\pi^{\theta(t)} \leq 2^{-128}$ и $(\delta_\pi)^{2\theta(t)} \leq 2^{-128}$ для подстановок из первой параметризации, имеющих характеристики $p_\pi = \frac{4}{128} = \frac{1}{32}$ и $\delta_\pi = \frac{15}{64}$. Система неравенств (1) для них примет вид:

$$\begin{cases} p_\pi^{\theta(t)} \leq 2^{-128} \\ (\delta_\pi)^{2\theta(t)} \leq 2^{-128} \end{cases} \Leftrightarrow \begin{cases} 5 \cdot \theta(t) \geq 128 \\ (6 - \log_2 15) \cdot 2\theta(t) \geq 128 \end{cases} \Leftrightarrow \begin{cases} \theta(t) \geq 25,6 \\ \theta(t) \geq 30,58 \end{cases}$$

Что опять же дает нам минимальное количество итераций, на котором любое разностное (линейное) соотношение не смогут быть различаемыми на всем материале в 2^{128} , равным 4. Таким образом, использование в алгоритме блочного шифрования «Кузнечик» подстановок из первой параметризации со значениями $p_\pi = \frac{4}{128}$ и $\delta_\pi = \frac{15}{64}$ не ухудшает его стойкость относительно разностного и линейного методов.

Выводы

Проведенное исследование принесло свои плоды, теперь известно, что подстановки на основе ARX-конструкций являются конкурентоспособным решением. При увеличении количества итераций достойные внимания подстановки встречаются все реже, но и характеристики постепенно улучшаются. На некоторых итерациях по значениям, например, разностной характеристики достигалось плато, но оно держалось лишь пару итераций, после чего характеристика улучшалась. Этот факт дает повод обоснованно предположить, что на следующих итерациях могут произойти улучшения характеристик, что позволит сначала найти подстановки с теми же характеристиками, что и у ГОСТ «Кузнечик», и далее на итерациях $T \in [13,16]$ найти параметризации, чьи криптографические характеристики будут совпадать с характеристиками S-Box-подстановки шифра AES. Более точная оценка затруднительна, так как невозможно экстраполировать текущие результаты на дальнейшие возможные исследования для большего количества итераций.

Целью исследования была оценка возможности построения подстановок с помощью ARX-конструкций с криптографическими характеристиками близкими к S-Box-подстановке шифра AES. Полученные результаты, а точнее подстановки с характеристиками почти совпадающими с ГОСТ «Кузнечик», указывают на достижение цели, так как среди рассмотренных параметризаций были найдены подстановки, чьи свойства лишь в 2 раза хуже, чем у AES. Результаты так же указывают, что подобный метод построения достоин внимания, потому как он сочетает возможности использования в низкоресурсной криптографии с неплохими характеристиками. Также показано, что найденные в работе подстановки из первой и второй параметризаций при их использовании в качестве нелинейного преобразования в алгоритме «Кузнечик» не уменьшают его стойкость

относительно разностного и линейного методов криптоанализа по сравнению с оригинальной подстановкой, используемой в алгоритме «Кузнечик».

Результатом исследования стали подстановки с вычисленными характеристиками, представленные в отдельных файлах для каждой T ; отдельные файлы с гистограммами для каждой T ; 3 варианта программы вычислений характеристик, листинги которых представлены в приложении, а также скрипт на языке Python для валидации результатов, представленный в Листинге 4.

Библиографический список

- [1]. J. Daemen and V. Rijmen, The Design of Rijndael: AES — The Advanced Encryption Standard, Springer, 2002.
- [2]. National Institute of Standards and Technology, FIPS PUB 197: Advanced Encryption Standard (AES), Nov. 2001.
- [3]. Федеральное агентство по техническому регулированию и метрологии Российской Федерации, ГОСТ Р 34.12-2015 «Информационная технология. Криптографическая защита информации. Блочные шифры», Москва, 2015.
- [4]. P. Rogaway and T. Shrimpton, “Cryptographic hash-function basics: Definitions, implications, and separations for preimage resistance, second-preimage resistance, and collision resistance,” in Fast Software Encryption, Lecture Notes in Computer Science, vol. 3017, Springer, 2004, pp. 371–388.
- [5]. P. Kocher, “Timing attacks on implementations of Diffie-Hellman, RSA, DSS, and other systems,” in Advances in Cryptology — CRYPTO ’96, Lecture Notes in Computer Science, vol. 1109, Springer, 1996, pp. 104–113.
- [6]. J. A. Halderman et al., “Lest we remember: Cold boot attacks on encryption keys,” in Proceedings of the 17th USENIX Security Symposium, 2008, pp. 45–60.
- [7]. E. Biham and A. Shamir, “Differential cryptanalysis of DES-like cryptosystems,” Journal of Cryptology, vol. 4, no. 1, pp. 3–72, 1991.
- [8]. M. Matsui, “Linear cryptanalysis method for DES cipher,” in Advances in Cryptology — EUROCRYPT ’93, Lecture Notes in Computer Science, vol. 765, Springer, 1994, pp. 386–397.
- [9]. R. E. Blahut, Fast Algorithms for Signal Processing, Cambridge University Press, 2010
- [10]. F. Chabaud and S. Vaudenay, “Links Between Differential and Linear Cryptanalysis,” in Advances in Cryptology — EUROCRYPT ’94, Lecture Notes in Computer Science, vol. 950, Springer, 1995, pp. 356–365.

- [11]. A. Courtois and J. Pieprzyk, “Cryptanalysis of block ciphers with overdefined systems of equations,” in *Advances in Cryptology — ASIACRYPT 2002*, *Lecture Notes in Computer Science*, vol. 2501, Springer, 2002, pp. 267–287.
- [12]. OpenMP Architecture Review Board, *OpenMP Application Programming Interface Version 5.1*, Nov. 2020.
- [13]. G. M. Amdahl, “Validity of the single processor approach to achieving large scale computing capabilities,” in *Proceedings of the AFIPS Spring Joint Computer Conference*, vol. 30, 1967, pp. 483–485.
- [14]. J. L. Gustafson, “Reevaluating Amdahl’s law,” *Communications of the ACM*, vol. 31, no. 5, pp. 532–533, 1988.
- [15]. S. Williams, A. Waterman, and D. Patterson, “Roofline: an insightful visual performance model for multicore architectures,” *Communications of the ACM*, vol. 52, no. 4, pp. 65–76, 2009.
- [16]. NVIDIA Corporation, *CUDA C Programming Guide*, 2024
- [17]. Малышев, Ф.М., Трифионов, Д.И. Рассеивающие свойства *XSLP*-шифров. Математические вопросы криптографии. – М.: Академия криптографии Российской Федерации, Математический институт им. В.А. Стеклова РАН. – 2016. – том 7, выпуск 3. – С. 47-60.
- [18]. Daemen, J., Rijmen, V.: *The Wide Trail Design Strategy*. In: *Cryptography and Coding: 8th IMA International Conference Cirencester, UK, December 17–19, 2001 Proceedings*. Springer Berlin Heidelberg, Berlin, Heidelberg (2001), pp. 222–238.
- [19]. Ерохин, А.В., Малышев, Ф.М., Тришин, А.Е., Многомерный линейный метод и показатели рассеивания линейной среды шифрпреобразований. Математические вопросы криптографии. – М.: Академия криптографии Российской Федерации, Математический институт им. В.А. Стеклова РАН. – 2017. – том 8, выпуск 4. – С. 29-62.
- [20]. Малышев, Ф.М. Двойственность разностного и линейного методов в криптографии. Математические вопросы криптографии. – М.: Академия

криптографии Российской Федерации, Математический институт им.
В.А. Стеклова РАН. – 2014. – том 5, выпуск 3. – С. 35-47.

Приложение

Листинг 1. Эталонная программа на языке C. (etalon.c)

```
#include <stdio.h>
#include <stdint.h>
#include <string.h>
#include <math.h>

static inline uint8_t cls4(uint8_t v, uint8_t shift) {
    shift &= 3; // mod 4
    uint8_t x = (v << shift) & 0x0F;
    uint8_t y = (v >> (4 - shift)) & 0x0F;
    return (x | y) & 0x0F;
}

static inline uint8_t A_single(uint8_t X, uint8_t r, uint8_t S) {
    uint8_t X1 = (X >> 4) & 0x0F;
    uint8_t X2 = X & 0x0F;

    uint8_t Z = (X1 + cls4(X2, r)) & 0x0F;
    uint8_t Y2 = (X2 ^ cls4(Z, S)) & 0x0F;

    return (uint8_t)((Z << 4) | Y2);
}

static uint8_t A_rounds(uint8_t X, const uint8_t *r, const uint8_t *S,
uint8_t T) {
    uint8_t Y = X;
    for (uint8_t i = 0; i < T; ++i) {
        Y = A_single(Y, r[i], S[i]);
    }
    return Y;
}

static void build_sbox(uint8_t sbox[256], const uint8_t *r, const uint8_t *S,
uint8_t T) {
    for (int x = 0; x < 256; ++x) {
        sbox[x] = A_rounds((uint8_t)x, r, S, T);
    }
}

static int is_permutation(const uint8_t sbox[256]) {
    uint8_t used[256];
    memset(used, 0, sizeof(used));
    for (int x = 0; x < 256; ++x) {
        if (used[sbox[x]]) {
            return 0;
        }
        used[sbox[x]] = 1;
    }
    return 1;
}

static float compute_p_pi(const uint8_t Y[256]) {
    int counter[256][256] = {0};
```

```

float max_prob = 0.0f;

for (int x = 0; x < 256; x++) {
    for (int a = 1; a < 256; a++) {
        int x_prime = x ^ a;
        int eps = Y[x] ^ Y[x_prime];
        if (eps == 0) continue;
        counter[a][eps]++;
    }
}

for (int a = 1; a < 256; a++) {
    int total = 0;
    for (int eps = 1; eps < 256; eps++) total += counter[a][eps];
    if (total == 0) continue;

    for (int eps = 1; eps < 256; eps++) {
        float prob = (float)counter[a][eps] / (float)total;
        if (prob > max_prob) max_prob = prob;
    }
}
return max_prob;
}

static int anf_degree(uint8_t *F) {
    uint8_t coeffs[256];
    memcpy(coeffs, F, 256);

    for (int i = 0; i < 8; i++) {
        for (int mask = 0; mask < 256; mask++) {
            if (mask & (1 << i)) {
                coeffs[mask] ^= coeffs[mask ^ (1 << i)];
            }
        }
    }

    int deg = 0;
    for (int mask = 1; mask < 256; mask++) {
        if (coeffs[mask]) {
            int d = __builtin_popcount(mask);
            if (d > deg) deg = d;
        }
    }
    return deg;
}

static int compute_lambda(uint8_t *Y) {
    int lambda = 8;
    uint8_t Yinv[256];

    for (int x = 0; x < 256; x++) {
        Yinv[Y[x]] = x;
    }

    for (int a = 1; a < 256; a++) {
        uint8_t f[256], g[256];

        for (int x = 0; x < 256; x++) {
            f[x] = __builtin_parity(a & Y[x]);
        }
        for (int x = 0; x < 256; x++) {
            g[x] = __builtin_parity(a & Yinv[x]);
        }
    }
}

```

```

    int deg_f    = anf_degree(f);
    int deg_g    = anf_degree(g);
    int max_deg  = (deg_f > deg_g ? deg_f : deg_g);

    if (max_deg < lambda) lambda = max_deg;
}
return lambda;
}

static void fwt_int_256(int f[256]) {
    for (int len = 1; len < 256; len <= 1) {
        for (int start = 0; start < 256; start += (len < 1)) {
            for (int i = 0; i < len; i++) {
                int a = f[start + i];
                int b = f[start + i + len];
                f[start + i]      = a + b;
                f[start + i + len] = a - b;
            }
        }
    }
}

static float compute_delta(const uint8_t Y[256]) {
    const float norm = 1.0f / 256.0f;
    float max_abs_delta = 0.0f;

    for (int beta = 1; beta < 256; beta++) {
        int f[256];

        for (int x = 0; x < 256; x++) {
            f[x] = __builtin_parity(beta & Y[x]) ? -1 : 1;
        }

        fwt_int_256(f);

        for (int alpha = 1; alpha < 256; alpha++) {
            float delta_alpha_beta = -(float)f[alpha] * norm;
            float abs_delta = delta_alpha_beta < 0 ? -delta_alpha_beta :
delta_alpha_beta;
            if (abs_delta > max_abs_delta) max_abs_delta = abs_delta;
        }
    }
    return max_abs_delta;
}

int main(void) {
    uint8_t T = 9;
    uint8_t r[10] = {2, 0, 3, 3, 2, 1, 3, 3, 2};
    uint8_t S[10] = {1, 2, 0, 3, 0, 0, 0, 0, 0};

    uint8_t sbbox[256];
    build_sbbox(sbbox, r, S, T);

    if (!is_permutation(sbbox)) {
        printf("Not a permutation for T=%u\n", T);
        return 0;
    }

    float p_pi      = compute_p_pi(sbbox);
    int    lambda_pi = compute_lambda(sbbox);

```

```

float delta_pi = compute_delta(sbox);

printf("T = %u\n", T);
printf("r = [");
for (int i = 0; i < T; ++i) printf("%u%s", r[i], (i+1<T)?", ":"");
printf("]\n");
printf("S = [");
for (int i = 0; i < T; ++i) printf("%u%s", S[i], (i+1<T)?", ":"");
printf("]\n");

printf("p_pi      = %.7f\n", p_pi);
printf("lambda_pi = %d\n",  lambda_pi);
printf("delta_pi   = %.7f\n", delta_pi);

return 0;
}

```

Листинг 2. Программа, оптимизированная с помощью SIMD и OpenMP (etalon_opt.c)

```
#include <stdio.h>
#include <stdint.h>
#include <time.h>
#include <string.h>
#include <math.h>
#include <stdbool.h>
#include <immintrin.h>
#include <omp.h>
#include <stdalign.h>

#define TOP_SIZE 5
#define MAX_T 10

typedef struct {
    float prob;
    int lambda_pi;
    float linear_bias;
    uint8_t r[MAX_T];
    uint8_t S[MAX_T];
} Combination;

static uint8_t cls_table[16][4];
static uint8_t A_single_cache[16][16][4][4];
static bool tables_initialized = false;

void init_tables() {
    if (tables_initialized) return;

    for (uint8_t val = 0; val < 16; ++val) {
        for (uint8_t shift = 0; shift < 4; ++shift) {
            cls_table[val][shift] = ((val << shift) | ((val << shift) >> 4))
& 0x0F;
        }
    }

    for (uint8_t X1 = 0; X1 < 16; ++X1) {
        for (uint8_t X2 = 0; X2 < 16; ++X2) {
            for (uint8_t r = 0; r < 4; ++r) {
                for (uint8_t S = 0; S < 4; ++S) {
                    uint8_t Z = (X1 + cls_table[X2][r]) & 0x0F;
                    uint8_t Y2 = (X2 ^ cls_table[Z][S]) & 0x0F;
                    A_single_cache[X1][X2][r][S] = (Z << 4) | Y2;
                }
            }
        }
    }
    tables_initialized = true;
}

static inline uint8_t A_single_opt(const uint8_t X, const uint8_t r, const
uint8_t S) {
    return A_single_cache[(X >> 4) & 0x0F][X & 0x0F][r][S];
}

static inline int is_permutation_opt(const uint8_t *sbox) {
```

```

uint64_t masks[4] = {0};
for (int i = 0; i < 256; ++i) {
    uint8_t val = sbbox[i];
    uint64_t bit = 1ULL << (val & 0x3F);
    int idx = val >> 6;
    if (masks[idx] & bit) return 0;
    masks[idx] |= bit;
}
return 1;
}

static inline uint8_t A(const uint8_t X, const uint8_t *r, const uint8_t *S,
const uint8_t T) {
    uint8_t Y = A_single_opt(X, r[0], S[0]);
    for (int i = 1; i < T; ++i)
        Y = A_single_opt(Y, r[i], S[i]);
    return Y;
}

static void compute_sbox(uint8_t *sbox, const uint8_t T, const uint8_t *r,
const uint8_t *S) {
    #pragma omp simd
    for (uint16_t x = 0; x < 256; ++x)
        sbox[x] = A(x, r, S, T);
}

static float compute_differential_prob_opt(const uint8_t *sbox) {
    alignas(32) uint16_t P[256][256] = {0};
    float max_prob = 0.0f;

    #pragma omp parallel for reduction(max:max_prob)
    for (int a = 1; a < 256; ++a) {
        uint16_t local_P[256] = {0};
        for (int x = 0; x < 256; ++x) {
            int x_xor_a = x ^ a;
            local_P[sbox[x] ^ sbox[x_xor_a]]++;
        }

        for (int b = 0; b < 256; ++b) {
            float prob = (float)local_P[b] / 256.0f;
            if (prob > max_prob) max_prob = prob;
        }
    }
    return max_prob;
}

static uint8_t parity(uint8_t x) {
    x ^= x >> 4;
    x ^= x >> 2;
    x ^= x >> 1;
    return x & 1;
}

static int count_bits(uint8_t x) {
    x = (x & 0x55) + ((x >> 1) & 0x55);
    x = (x & 0x33) + ((x >> 2) & 0x33);
    return (x & 0x0F) + (x >> 4);
}

uint8_t compute_lambda_pi_opt(const uint8_t sbox[256]) {
    uint8_t min_degree = 8;

```

```

#pragma omp parallel for reduction(min:min_degree)
for (uint16_t alpha = 1; alpha < 256; ++alpha) {
    uint8_t anf[256] = {0};
    for (int x = 0; x < 256; ++x)
        anf[x] = parity(alpha & sbox[x]);

    for (int i = 0; i < 8; ++i)
        for (int j = 0; j < 256; ++j)
            if (j & (1 << i))
                anf[j] ^= anf[j ^ (1 << i)];

    uint8_t current_degree = 0;
    for (int j = 0; j < 256; ++j) {
        if (anf[j]) {
            int bits = count_bits(j);
            current_degree = bits > current_degree ? bits :
current_degree;
        }
    }

    if (current_degree < min_degree)
        min_degree = current_degree;
}
return min_degree;
}

static float compute_delta_opt(const uint8_t sbox[256]) {
    alignas(32) int f[256];
    float max_delta = 0.0f;

    #pragma omp parallel for reduction(max:max_delta)
    for (int beta = 1; beta < 256; beta++) {
        for (int x = 0; x < 256; x++) {
            f[x] = (parity(beta & sbox[x]) == 0) ? 1 : -1;
        }

        int len = 128;
        while (len > 0) {
            for (int start = 0; start < 256; start += 2 * len) {
                for (int i = start; i < start + len; i++) {
                    int a = f[i];
                    int b = f[i + len];
                    f[i] = a + b;
                    f[i + len] = a - b;
                }
            }
            len >>= 1;
        }

        for (int alpha = 1; alpha < 256; alpha++) {
            float bias = fabsf((float)f[alpha]) / 256.0f;
            if (bias > max_delta) {
                max_delta = bias;
            }
        }
    }
    return max_delta;
}

static int is_pareto_dominant(const Combination *a, const Combination *b) {
    return (a->prob <= b->prob) &&

```

```

        (a->linear_bias <= b->linear_bias) &&
        (a->lambda_pi >= b->lambda_pi) &&
        ((a->prob < b->prob) ||
         (a->linear_bias < b->linear_bias) ||
         (a->lambda_pi > b->lambda_pi));
    }

void update_pareto_front(Combination *front, int *count, Combination
candidate) {
    bool dominated = false;

    #pragma omp critical
    {
        for (int i = 0; i < *count; ++i) {
            if (is_pareto_dominant(&front[i], &candidate)) {
                dominated = true;
                break;
            }
        }

        if (!dominated) {
            Combination new_front[TOP_SIZE];
            int new_count = 0;

            for (int i = 0; i < *count; ++i) {
                if (!is_pareto_dominant(&candidate, &front[i])) {
                    new_front[new_count++] = front[i];
                }
            }

            if (new_count < TOP_SIZE) {
                new_front[new_count++] = candidate;
            }

            memcpy(front, new_front, new_count * sizeof(Combination));
            *count = new_count;
        }
    }
}

void process_all_combinations(uint8_t T) {
    init_tables();
    char filename[26];
    snprintf(filename, sizeof(filename), "Combinations_T%d.txt", T);
    FILE *file = fopen(filename, "w");
    if (!file) { perror("Error"); return; }

    const uint64_t total_combinations = 1ULL << (4 * T);
    Combination pareto_front[TOP_SIZE] = {{0}};
    int pareto_count = 0;

    time_t start_time = time(NULL);
    fprintf(file, "Start time: %s", ctime(&start_time));
    printf("T=%d (%llu comb) | Threads: %d\n", T, total_combinations,
omp_get_max_threads());

    #pragma omp parallel for schedule(dynamic)
    for (uint64_t combo = 0; combo < total_combinations; ++combo) {
        uint8_t r[MAX_T], S[MAX_T];
        uint8_t sbox[256];
        uint64_t tmp = combo;

```

```

    for (int i = 0; i < T; ++i) {
        r[i] = tmp & 0x03;
        tmp >>= 2;
    }
    for (int i = 0; i < T; ++i) {
        S[i] = tmp & 0x03;
        tmp >>= 2;
    }

    compute_sbox(sbox, T, r, S);
    if (!is_permutation_opt(sbox)) continue;

    Combination current;
    current.prob = compute_differential_prob_opt(sbox);
    current.lambda_pi = compute_lambda_pi_opt(sbox);
    current.linear_bias = compute_delta_opt(sbox);
    memcpy(current.r, r, T);
    memcpy(current.S, S, T);

    update_pareto_front(pareto_front, &pareto_count, current);

    if (combo % (1ULL << 20) == 0) {
        #pragma omp critical
        printf("\rProgress: %.2f%%", (double)combo / total_combinations *
100);
    }
}

// Sort Pareto front
for (int i = 0; i < pareto_count; ++i) {
    for (int j = i+1; j < pareto_count; ++j) {
        if ((pareto_front[i].prob > pareto_front[j].prob) ||
            (pareto_front[i].prob == pareto_front[j].prob &&
             pareto_front[i].lambda_pi < pareto_front[j].lambda_pi)) {
            Combination tmp = pareto_front[i];
            pareto_front[i] = pareto_front[j];
            pareto_front[j] = tmp;
        }
    }
}

fprintf(file, "=== Pareto optimal combinations (T=%d) ===\n", T);
for (int i = 0; i < pareto_count; ++i) {
    fprintf(file, "r = [");
    for (int j = 0; j < T; ++j)
        fprintf(file, "%d ", pareto_front[i].r[j]);
    fprintf(file, "], S = [");
    for (int j = 0; j < T; ++j)
        fprintf(file, "%d ", pareto_front[i].S[j]);
    fprintf(file, "], p_pi = %.7f, lambda_pi = %d, delta_pi = %.7f\n",
            pareto_front[i].prob,
            pareto_front[i].lambda_pi,
            pareto_front[i].linear_bias);
}

time_t end_time = time(NULL);
fprintf(file, "\nTotal time: %.2f seconds\n", difftime(end_time,
start_time));
fclose(file);
printf("\nResults saved to %s\n", filename);

```

```
}  
  
int main() {  
    uint8_t T;  
    printf("Enter T: ");  
    if (scanf("%hhu", &T) != 1 || T < 1 || T > 10) {  
        printf("Invalid T\n");  
        return 1;  
    }  
    process_all_combinations(T);  
    return 0;  
}
```

Листинг 3. Программа, реализованная на CUDA. (cuda_version.cu)

```
#define NOMINMAX

#include <cstdio>
#include <cstdlib>
#include <stdlib.h>
#include <cstring>
#include <cmath>
#include <ctime>
#include <vector>
#include <algorithm>
#include <string>
#include <fstream>
#include <iostream>
#include <iomanip>
#include <sstream>
#include <map>

#ifdef _WIN32
#include <windows.h>
#else
#include <unistd.h>
#include <fcntl.h>
#endif

#define TOP_SIZE 20
#define MAX_T 20

#define METRIC_PPI_ONLY 0
#define METRIC_DELTA_ONLY 1
#define METRIC_ALL 2
#define METRIC_HIST 3
#define METRIC_ALL_HIST 4

struct Combination {
    float p_pi;
    int lambda_pi;
    float delta_pi;
    uint8_t r[MAX_T];
    uint8_t S[MAX_T];
};

struct DeviceOut {
    float *p_pi;
    int *lambda_pi;
    float *delta_pi;
    uint8_t *rS; // [iters][2*MAX_T]
    uint8_t *is_valid; // [iters]
};

// Для histogram: ключ = (ddt_max, lambda, lat_max)
struct HistKey {
    uint16_t ddt_max; // raw DDT max
    int lambda; // algebraic degree
    uint16_t lat_max; // raw LAT max
};
```

```

    bool operator<(const HistKey &o) const {
        if (ddt_max != o.ddt_max) return ddt_max < o.ddt_max;
        if (lambda != o.lambda) return lambda < o.lambda;
        return lat_max < o.lat_max;
    }
};

__constant__ uint8_t d_A_single_cache[16][16][4][4];

static uint8_t h_A_single_cache[16][16][4][4];

static void init_tables_host() {
    static bool initied = false;
    if (initied) return;

    uint8_t cls_table[16][4];
    for (uint8_t val = 0; val < 16; ++val)
        for (uint8_t shift = 0; shift < 4; ++shift)
            cls_table[val][shift] = (uint8_t)((val << shift) | ((val <<
shift) >> 4)) & 0x0F);

    for (uint8_t X1 = 0; X1 < 16; ++X1)
        for (uint8_t X2 = 0; X2 < 16; ++X2)
            for (uint8_t r = 0; r < 4; ++r)
                for (uint8_t S = 0; S < 4; ++S) {
                    uint8_t Z = (uint8_t)((X1 + cls_table[X2][r]) & 0x0F);
                    uint8_t Y2 = (uint8_t)((X2 ^ cls_table[Z][S]) & 0x0F);
                    h_A_single_cache[X1][X2][r][S] = (uint8_t)((Z << 4) |
Y2);
                }

    initied = true;
}

static void upload_tables_to_device() {
    init_tables_host();
    cudaMemcpyToSymbol(d_A_single_cache, h_A_single_cache,
sizeof(h_A_single_cache));
}

__device__ __forceinline__ uint8_t A_single_opt_dev(uint8_t X, uint8_t r,
uint8_t S) {
    return d_A_single_cache[(X >> 4) & 0x0F][X & 0x0F][r][S];
}

__device__ __forceinline__ uint8_t A_dev(uint8_t X, const uint8_t *r, const
uint8_t *S, uint8_t T) {
    uint8_t Y = A_single_opt_dev(X, r[0], S[0]);
    #pragma unroll
    for (int i = 1; i < MAX_T; ++i) {
        if (i >= T) break;
        Y = A_single_opt_dev(Y, r[i], S[i]);
    }
    return Y;
}

__device__ __forceinline__ int is_permutation_dev(const uint8_t sbox[256]) {
    uint64_t masks[4] = {0,0,0,0};
    #pragma unroll 1
    for (int i = 0; i < 256; ++i) {

```

```

        uint8_t val = sbbox[i];
        uint64_t bit = 1ULL << (val & 0x3F);
        int idx = val >> 6;
        if (masks[idx] & bit) return 0;
        masks[idx] |= bit;
    }
    return 1;
}

__device__ __forceinline__ uint8_t parity8(uint8_t x) {
    x ^= x >> 4; x ^= x >> 2; x ^= x >> 1;
    return x & 1;
}

__device__ __forceinline__ int popcnt8(uint8_t x) {
    x = (x & 0x55) + ((x >> 1) & 0x55);
    x = (x & 0x33) + ((x >> 2) & 0x33);
    return (x & 0x0F) + (x >> 4);
}

__device__ uint16_t compute_ddt_max_dev(const uint8_t sbbox[256]) {
    uint16_t global_max = 0, row[256];
    for (int a = 1; a < 256; ++a) {
        #pragma unroll 1
        for (int i = 0; i < 256; ++i) row[i] = 0;
        #pragma unroll 1
        for (int x = 0; x < 256; ++x) {
            uint8_t b = sbbox[x] ^ sbbox[x ^ a];
            row[b]++;
        }
        #pragma unroll 1
        for (int b = 0; b < 256; ++b)
            if (row[b] > global_max) global_max = row[b];
    }
    return global_max;
}

__device__ int16_t compute_lat_max_abs_dev(const uint8_t sbbox[256]) {
    int16_t max_abs = 0, f[256];
    for (int beta = 1; beta < 256; ++beta) {
        for (int x = 0; x < 256; ++x)
            f[x] = (parity8((uint8_t)(beta & sbbox[x])) == 0) ? 1 : -1;
        for (int len = 1; len < 256; len <= 1)
            for (int i = 0; i < 256; i += (len < 1))
                for (int j = 0; j < len; ++j) {
                    int16_t u = f[i+j], v = f[i+j+len];
                    f[i+j] = (int16_t)(u + v);
                    f[i+j+len] = (int16_t)(u - v);
                }
        for (int alpha = 1; alpha < 256; ++alpha) {
            int16_t v = f[alpha]; if (v < 0) v = -v;
            if (v > max_abs) max_abs = v;
        }
    }
    return max_abs;
}

__device__ int compute_lambda_min_degree_dev(const uint8_t sbbox[256]) {
    uint8_t coord[8][256];
    for (int x = 0; x < 256; ++x) {
        uint8_t y = sbbox[x];
        #pragma unroll

```

```

        for (int i = 0; i < 8; ++i) coord[i][x] = (y >> i) & 1;
    }
    auto mobius = [&] (uint8_t f[256]) {
        for (int i = 0; i < 8; ++i)
            for (int m = 0; m < 256; ++m)
                if (m & (1 << i)) f[m] ^= f[m ^ (1 << i)];
    };
    for (int i = 0; i < 8; ++i) mobius(coord[i]);

    uint64_t bits[8][9][4] = {};
    for (int i = 0; i < 8; ++i)
        for (int m = 0; m < 256; ++m) if (coord[i][m]) {
            int d = popcnt8((uint8_t)m);
            bits[i][d][m >> 6] |= 1ULL << (m & 63);
        }

    int min_deg = 8;
    for (int a = 1; a < 256; ++a) {
        uint64_t acc[9][4] = {};
        for (int i = 0; i < 8; ++i) if (a & (1 << i))
            for (int d = 0; d <= 8; ++d)
                for (int w = 0; w < 4; ++w)
                    acc[d][w] ^= bits[i][d][w];

        int cur_deg = 0;
        for (int d = 8; d >= 0; --d) {
            uint64_t orr = acc[d][0] | acc[d][1] | acc[d][2] | acc[d][3];
            if (orr) { cur_deg = d; break; }
        }
        if (cur_deg < min_deg) min_deg = cur_deg;
    }
    return min_deg;
}

__global__ void enumerateKernel(
    uint8_t T, uint64_t start_index, uint64_t count, int metric_mode,
    DeviceOut out)
{
    uint64_t gid = blockIdx.x * blockDim.x + threadIdx.x;
    uint64_t gdim = gridDim.x * (uint64_t)blockDim.x;

    for (uint64_t off = gid; off < count; off += gdim) {
        uint64_t idx = start_index + off;
        uint8_t r[MAX_T], S[MAX_T], sbox[256];
        uint64_t tmp = idx;
        for (int i = 0; i < T; ++i) { r[i] = tmp & 0x03; tmp >>= 2; }
        for (int i = 0; i < T; ++i) { S[i] = tmp & 0x03; tmp >>= 2; }
        for (int x = 0; x < 256; ++x)
            sbox[x] = A_dev((uint8_t)x, r, S, T);

        if (!is_permutation_dev(sbox)) {
            out.is_valid[off] = 0;
            continue;
        }

        float p_pi_f = 1e30f;
        float delta_f = 1e30f;
        int lambdaMin = -1;

        if (metric_mode == METRIC_PPI_ONLY) {
            uint16_t ddt = compute_ddt_max_dev(sbox);

```

```

        p_pi_f = ddt / 256.0f;
    } else if (metric_mode == METRIC_DELTA_ONLY) {
        int16_t lat = compute_lat_max_abs_dev(sbox);
        delta_f = lat / 256.0f;
    } else {
        uint16_t ddt = compute_ddt_max_dev(sbox);
        int16_t lat = compute_lat_max_abs_dev(sbox);
        lambdaMin = compute_lambda_min_degree_dev(sbox);
        p_pi_f = ddt / 256.0f;
        delta_f = lat / 256.0f;
    }

    out.p_pi[off] = p_pi_f;
    out.delta_pi[off] = delta_f;
    out.lambda_pi[off] = lambdaMin;

    uint8_t *dst = out.rS + off * (2 * MAX_T);
    for (int i = 0; i < MAX_T; ++i) dst[i] = (i < T ?
r[i] : 0);
    for (int i = 0; i < MAX_T; ++i) dst[MAX_T + i] = (i < T ?
S[i] : 0);

    out.is_valid[off] = 1;
}
}

static inline bool pareto_dominates(const Combination &a, const Combination
&b) {
    bool better_or_equal = (a.p_pi <= b.p_pi) &&
        (a.delta_pi <= b.delta_pi) &&
        (a.lambda_pi >= b.lambda_pi);
    bool strictly_better = (a.p_pi < b.p_pi) ||
        (a.delta_pi < b.delta_pi) ||
        (a.lambda_pi > b.lambda_pi);
    return better_or_equal && strictly_better;
}

static void update_pareto_front(std::vector<Combination> &front,
                                const Combination &cand)
{
    for (auto &f : front)
        if (pareto_dominates(f, cand))
            return;
    std::vector<Combination> nf;
    nf.reserve(TOP_SIZE);
    for (auto &f : front)
        if (!pareto_dominates(cand, f))
            nf.push_back(f);
    if ((int)nf.size() < TOP_SIZE)
        nf.push_back(cand);
    front.swap(nf);
}

static void update_top_p(std::vector<Combination> &top_p,
                          const Combination &cand)
{
    auto it = std::lower_bound(
        top_p.begin(), top_p.end(), cand,
        [](auto &a, auto &b) { return a.p_pi < b.p_pi; }
    );
    if (it != top_p.end() || top_p.size() < TOP_SIZE) {
        top_p.insert(it, cand);
    }
}

```

```

        if (top_p.size() > TOP_SIZE)
            top_p.pop_back();
    }

static void update_top_delta(std::vector<Combination> &top_d,
                             const Combination &cand)
{
    auto it = std::lower_bound(
        top_d.begin(), top_d.end(), cand,
        [] (auto &a, auto &b) { return a.delta_pi < b.delta_pi; }
    );
    if (it != top_d.end() || top_d.size() < TOP_SIZE) {
        top_d.insert(it, cand);
        if (top_d.size() > TOP_SIZE)
            top_d.pop_back();
    }
}

static void write_results(
    const std::string &filename,
    const std::vector<Combination> &pareto,
    const std::vector<Combination> &top_p,
    const std::vector<Combination> &top_d,
    uint8_t T,
    time_t start, time_t end,
    unsigned long long checked)
{
    std::ofstream file(filename);
    if (!file) { std::perror("Error opening output file"); return; }

    auto fmt_float = [] (float v) -> std::string {
        if (!(v < 1e29f)) return "NA";
        std::ostringstream ss;
        ss << std::fixed << std::setprecision(7) << v;
        return ss.str();
    };

    file << "=== Pareto optimal combinations (T=" << int(T) << ") ===\n";
    if (pareto.empty()) {
        file << "(none found)\n";
    } else {
        for (auto &c : pareto) {
            file << "r=[";
            for (int j = 0; j < T; ++j) file << int(c.r[j]) << " ";
            file << "], S=[";
            for (int j = 0; j < T; ++j) file << int(c.S[j]) << " ";
            file << "], p_pi=" << fmt_float(c.p_pi)
                << ", lambda_pi="
                << (c.lambda_pi >= 0 ? std::to_string(c.lambda_pi) : "NA")
                << ", delta_pi=" << fmt_float(c.delta_pi) << "\n";
        }
    }

    file << "\n=== Top " << TOP_SIZE << " minimal p_pi (with r/S) ===\n";
    if (top_p.empty()) {
        file << "(none found)\n";
    } else {
        for (auto &c : top_p) {
            file << "r=[";

```

```

        for (int j = 0; j < T; ++j) file << int(c.r[j]) << " ";
        file << "], S=[";
        for (int j = 0; j < T; ++j) file << int(c.S[j]) << " ";
        file << "], p_pi=" << fmt_float(c.p_pi)
            << ", lambda_pi="
            << (c.lambda_pi >= 0 ? std::to_string(c.lambda_pi) : "NA")
            << ", delta_pi=" << fmt_float(c.delta_pi) << "\n";
    }
}

file << "\n=== Top " << TOP_SIZE << " minimal delta_pi (with r/S) ===\n";
if (top_d.empty()) {
    file << "(none found)\n";
} else {
    for (auto &c : top_d) {
        file << "r=[";
        for (int j = 0; j < T; ++j) file << int(c.r[j]) << " ";
        file << "], S=[";
        for (int j = 0; j < T; ++j) file << int(c.S[j]) << " ";
        file << "], p_pi=" << fmt_float(c.p_pi)
            << ", lambda_pi="
            << (c.lambda_pi >= 0 ? std::to_string(c.lambda_pi) : "NA")
            << ", delta_pi=" << fmt_float(c.delta_pi) << "\n";
    }
}

file << "\n---\n";
file << "Checked combinations: " << checked << "\n";
file << "Start time: " << ctime(&start);
file << "End time: " << ctime(&end);
file << "Elapsed: " << std::fixed << std::setprecision(2)
    << difftime(end, start) << " sec\n";
std::cout << "Results saved to " << filename << "\n";
}

static void write_histogram(
    const std::string &filename,
    const std::map<HistKey, uint64_t> &hist,
    uint64_t total,
    uint8_t T,
    time_t start, time_t end)
{
    std::ofstream file(filename);
    if (!file) { std::perror("Error opening histogram file"); return; }

    file << "=== Histogram of combinations (T=" << int(T) << ") ===\n";
    file << "p_pi\tlambda_pi\tdelta_pi\tcount\tpercent\n";
    for (auto &kv : hist) {
        float p = kv.first.ddt_max / 256.0f;
        int lam = kv.first.lambda;
        float d = kv.first.lat_max / 256.0f;
        uint64_t cnt = kv.second;
        double pct = 100.0 * cnt / (double)total;
        file << std::fixed << std::setprecision(7)
            << p << "\t"
            << lam << "\t"
            << d << "\t"
            << cnt << "\t"
            << std::fixed << std::setprecision(4)
            << pct << "%\n";
    }
    file << "---\n";
}

```

```

file << "Total valid combinations: " << total << "\n";
file << "Start time: " << ctime(&start);
file << "End time: " << ctime(&end);
file << "Elapsed: " << std::fixed << std::setprecision(2)
    << difftime(end, start) << " sec\n";
std::cout << "Histogram saved to " << filename << "\n";
}

static void run_full_enumeration(uint8_t T, int metric_mode) {
    upload_tables_to_device();

    if (4ULL * T >= 64) {
        return;
    }

    const uint64_t total = 1ULL << (4 * T);
    const uint64_t chunk = 1ULL << 22;

    dim3 block(128);
    dim3 grid(static_cast<unsigned int>(
        std::min<uint64_t>((chunk + block.x - 1) / block.x, 65535ULL)
    ));

    std::vector<uint8_t> h_valid(chunk);
    std::vector<float> h_p(chunk), h_d(chunk);
    std::vector<int> h_l(chunk);
    std::vector<uint8_t> h_rS(chunk * 2 * MAX_T);

    DeviceOut d{};
    cudaMalloc(&d.p_pi, chunk * sizeof(float));
    cudaMalloc(&d.lambda_pi, chunk * sizeof(int));
    cudaMalloc(&d.delta_pi, chunk * sizeof(float));
    cudaMalloc(&d.rS, chunk * 2 * MAX_T * sizeof(uint8_t));
    cudaMalloc(&d.is_valid, chunk * sizeof(uint8_t));

    std::vector<Combination> pareto, top_p, top_d;
    std::map<HistKey, uint64_t> hist;
    uint64_t total_checked = 0;
    time_t start = time(NULL);

    for (uint64_t start_idx = 0; start_idx < total; start_idx += chunk) {
        uint64_t count = std::min<uint64_t>(chunk, total - start_idx);

        cudaMemset(d.is_valid, 0, count * sizeof(uint8_t));

        enumerateKernel<<<grid, block>>>>(T, start_idx, count, metric_mode,
d);

        cudaDeviceSynchronize();

        cudaMemcpy(h_valid.data(), d.is_valid, count * sizeof(uint8_t),
cudaMemcpyDeviceToHost);
        cudaMemcpy(h_p.data(), d.p_pi, count * sizeof(float),
cudaMemcpyDeviceToHost);
        cudaMemcpy(h_l.data(), d.lambda_pi, count * sizeof(int),
cudaMemcpyDeviceToHost);
        cudaMemcpy(h_d.data(), d.delta_pi, count * sizeof(float),
cudaMemcpyDeviceToHost);
        cudaMemcpy(h_rS.data(), d.rS, count * 2 * MAX_T *
sizeof(uint8_t), cudaMemcpyDeviceToHost);

```

```

        for (uint64_t i = 0; i < count; ++i) {
            if (!h_valid[i]) continue;
            ++total_checked;
            Combination c{};
            c.p_pi = h_p[i];
            c.lambda_pi = h_l[i];
            c.delta_pi = h_d[i];
            for (int j = 0; j < T; ++j) {
                c.r[j] = h_rS[i*2*MAX_T + j];
                c.S[j] = h_rS[i*2*MAX_T + MAX_T + j];
            }

            bool do_pareto = (metric_mode == METRIC_ALL) || (metric_mode ==
METRIC_ALL_HIST) ||
                            (metric_mode == METRIC_PPI_ONLY) || (metric_mode
== METRIC_DELTA_ONLY);
            bool do_top_p = (metric_mode != METRIC_DELTA_ONLY);
            bool do_top_d = (metric_mode != METRIC_PPI_ONLY);

            if (metric_mode != METRIC_HIST) {
                if (do_pareto) update_pareto_front(pareto, c);
                if (do_top_p && metric_mode != METRIC_DELTA_ONLY)
update_top_p(top_p, c);
                if (do_top_d && metric_mode != METRIC_PPI_ONLY)
update_top_delta(top_d, c);
            }

            if (metric_mode == METRIC_HIST || metric_mode == METRIC_ALL_HIST)
{

                uint16_t p_raw = 0, d_raw = 0;
                if (c.p_pi < 1e29f) p_raw =
static_cast<uint16_t>(std::lround(c.p_pi * 256.0f));
                if (c.delta_pi < 1e29f) d_raw =
static_cast<uint16_t>(std::lround(c.delta_pi * 256.0f));
                HistKey key{ p_raw, c.lambda_pi, d_raw };
                hist[key]++;
            }
        }

        if (((start_idx / chunk) % 50) == 0) {
            double pct = 100.0 * (double)(start_idx + count) / (double)total;
            std::cerr << "\rProgress: "
                << std::fixed << std::setprecision(2)
                << pct << "%    " << std::flush;
        }
    }
    std::cerr << "\n";

    time_t end = time(NULL);
    cudaFree(d.p_pi);
    cudaFree(d.lambda_pi);
    cudaFree(d.delta_pi);
    cudaFree(d.rS);
    cudaFree(d.is_valid);

    char fname[128];
    if (metric_mode != METRIC_HIST && metric_mode != METRIC_ALL_HIST) {
        std::snprintf(fname, sizeof(fname), "combinations_T%u_cuda.txt",
(unsigned)T);
        write_results(fname, pareto, top_p, top_d, T, start, end,
total_checked);
    }

```

```

    } else if (metric_mode == METRIC_HIST) {
        std::snprintf(fname, sizeof(fname), "histogram_T%u_cuda.txt",
(unsigned)T);
        write_histogram(fname, hist, total_checked, T, start, end);
    } else {
        std::snprintf(fname, sizeof(fname), "combinations_T%u_cuda.txt",
(unsigned)T);
        write_results(fname, pareto, top_p, top_d, T, start, end,
total_checked);
        std::snprintf(fname, sizeof(fname), "histogram_T%u_cuda.txt",
(unsigned)T);
        write_histogram(fname, hist, total_checked, T, start, end);
    }
}

int main() {
    unsigned int T = 1;
    std::cout << "Enter T: ";
    if (!(std::cin >> T)) return 1;
    if (T < 1 || T > 20) { return 1; }

    int metric_mode = METRIC_ALL;
    std::cout << "Select metric mode (0 - p_pi only, 1 - delta_pi only, 2 -
all, 3 - histogram, 4 - all+hist): ";
    if (!(std::cin >> metric_mode)) return 1;
    if (metric_mode < METRIC_PPI_ONLY || metric_mode > METRIC_ALL_HIST) {
        std::cout << "Invalid metric mode\n";
        return 1;
    }

    run_full_enumeration((uint8_t)T, metric_mode);

    return 0;
}

```

Листинг 4. Скрипт на языке Python для валидации характеристик. (sage.py)

```
from sage.crypto.sbox import SBox
from sage.crypto.boolean_function import BooleanFunction

def cls(byte, shift):
    b = (byte & 0x0F) << shift
    return ((b | (b >> 4)) & 0x0F)

def generate_sbox(T, r, S, use_xor=True):
    sbox = []
    for x in range(256):
        y = x
        for i in range(T):
            X1 = (y >> 4) & 0x0F
            X2 = y & 0x0F
            Z = (X1 + cls(X2, r[i])) & 0x0F
            if use_xor:
                Y2 = (X2 ^ cls(Z, S[i])) & 0x0F
            else:
                Y2 = (X2 + cls(Z, S[i])) & 0x0F
            y = (Z << 4) | Y2
        sbox.append(y)
    return sbox

def compute_lambda_pi(sbox_obj):
    min_degree = 8
    for b in range(1, 256):
        bf = sbox_obj.component_function(b)
        deg = bf.algebraic_degree()
        if deg < min_degree:
            min_degree = deg
            if min_degree == 0:
                break
    return min_degree

def compute_delta_pi(sbox_obj):
    LAT = sbox_obj.linear_approximation_table()
    max_abs = 0
    for a in range(1, 256):
        for b in range(1, 256):
            v = abs(LAT[a][b])
            if v > max_abs:
                max_abs = v
    return 2 * float(max_abs) / 256.0

def check(T, r, S, p_pi=None, lambda_pi=None, delta_pi=None):
    sbox_list = generate_sbox(T, r, S)
    sbox = SBox(sbox_list)

    if not sbox.is_permutation():
        print(f"T={T} r={r} S={S} → Не биективна")
        return

    du = sbox.differential_uniformity()
    sage_prob = du / 256.0

    if p_pi is not None:
        match = abs(p_pi - sage_prob) < 1e-9
```

```

        result = "Совпадение" if match else "Расхождение"
        print(f"{result} | Sage p_pi: {sage_prob:.7f}, Ваша: {p_pi:.7f}")
    else:
        print(f"T={T} r={r} S={S} → DU={du} (p_pi={sage_prob:.7f})")

lp = compute_lambda_pi(sbox)
if lambda_pi is not None:
    match = abs(lambda_pi - lp) < 1e-9
    result = "Совпадение" if match else "Расхождение"
    print(f"{result} | Sage lambda_pi: {lp}, Ваша: {lambda_pi}")
else:
    print(f"(lambda_pi={lp})")

dp = compute_delta_pi(sbox)
if delta_pi is not None:
    match = abs(delta_pi - dp) < 1e-9
    result = "Совпадение" if match else "Расхождение"
    print(f"{result} | Sage delta_pi: {dp:.7f}, Ваша: {delta_pi:.7f}")
else:
    print(f"(delta_pi={dp:.7f})")

```

```

T = 9
r = [2, 0, 3, 3, 2, 1, 3, 3, 2]
S = [1, 2, 0, 3, 0, 0, 0, 0, 0]
p_pi = 0.0312500
lambda_pi = 7
delta_pi = 0.2343750
check(T, r, S, p_pi, lambda_pi, delta_pi)

```